

OYUN GELİŞTİRME

Yeni Başlayanlar İçin Terimler El Kitabı

Godot + GDScript odaklı • genel oyun geliştirme sözlüğü • ders formatı

14

DERS BÖLÜMÜ

300+

TERİM

14

MİNİ TEST

AMAÇ Bir kursta, videoda, dokümantasyonda veya geliştirici sohbetinde terim duyduğunda "o neydi?" dememen. Her şeyi uygulayabilmek zorunda değilsin; önce terimi doğru kategoriye yerleştir.

Odak: Godot 4.x + GDScript

Genel oyun geliştirme terimleri motor bağımsız anlatılır; Godot'a özgü yerlerde Godot isimleri kullanılır.

Bu PDF nasıl çalışılır?

1. TUR Önce sadece başlıkları ve “Aklında kalsın” satırlarını oku. Hedefin tanışmak.

2. TUR Her bölümde core terimleri kendi cümlele açıklamaya çalış. Kod örneklerini Godot'ta 2-3 dakikalık mini denemelere dönüştür.

3. TUR Mini kontrol sorularını kapalı kitap cevapla. Yanlıklarını işaretleyip yalnızca o kartlara geri dön.

Ezberlemen gerekenler

- İlk etapta her terimin detayını değil, hangi problem alanına ait olduğunu bil.
- Node / Scene / signal / delta / input / vector / collision / resource / state / debugging gibi temel kelimeleri aktif olarak kullanabilir hale gel.
- Networking, shader, CI/CD, behavior tree gibi ileri başlıklarda şimdilik “duyunca tanı” seviyesi yeterli.
- Terimi anlamamanın en iyi testi: 1 cümle tanım + 1 kullanım örneği verebiliyor musun?

Öncelik kodu

TEMEL	İlk aylarda aktif olarak anlamaman gereken terim.
YAKINDA	Birkaç küçük projeden sonra sık kullanacağın terim.
TANI	Şimdilik duyduğunda kategorisini bilmen yeterli.

İçindekiler

- 01 Motor, proje ve oyun geliştirme ekosistemi
- 02 Programlama ve GDScript temelleri
- 03 Godot'un kalbi: Node, Scene ve SceneTree
- 04 Game loop, frame, delta ve input
- 05 Koordinatlar, Vector ve oyun matematiği
- 06 Physics, collision ve hareket
- 07 Görsel, 2D/3D, kamera ve animasyon
- 08 UI, resolution ve responsive tasarım
- 09 Data, Resource, save/load ve mimari
- 10 Gameplay, AI ve sistem tasarımı terimleri
- 11 Audio, networking ve multiplayer sözlüğü
- 12 Debugging, performance ve üretim araçları
- 13 Git, proje düzeni, build ve yayınlama
- 14 Oyun tasarımı ve proje üretim dili
- 15 Alfabetik büyük sözlük + mini test cevapları + kaynaklar

BÖLÜM 01

Motor, proje ve oyun geliştirme ekosistemi

Bir kurs açtığında ilk karşılaşacağın büyük resim terimleri.

HEDEF Bu bölüm bittiğinde 8 ana terimi açıklayabilecek ve 10 ek terimi duyduğunda tanıyabileceksin.

Game Engine (Oyun Motoru) TEMEL

Grafik, input, fizik, ses, sahne yönetimi ve platforma çıktı alma gibi temel işleri hazır sunan geliştirme ortamıdır. Godot, Unity ve Unreal birer oyun motorudur.

Nerede duyarsın? "Hangi engine'i kullanıyorsun?", "engine API".

Aklında kalsın: Motor oyunun kendisi değildir; oyunu üretmek için kullandığın araç takımıdır.

Editor TEMEL

Motorun görsel çalışma alanıdır. Sahne düzenler, node ekler, Inspector'dan değer değiştirir, script açar ve oyunu çalıştırır.

Nerede duyarsın? Godot Editor, Unity Editor.

Aklında kalsın: Engine arka plandaki sistem; editor ise onunla çalıştığın arayüzdür.

Project TEMEL

Bir oyuna ait sahneler, scriptler, assetler ve ayarların tamamıdır. Godot'ta proje kökünde project.godot bulunur.

Nerede duyarsın? Project Settings, project folder.

Aklında kalsın: Tek bir oyun = bir proje diye düşün.

Asset TEMEL

Oyunda kullanılan içerik dosyalarının genel adıdır: görsel, ses, font, 3D model, animasyon, veri dosyası vb.

Nerede duyarsın? asset pack, asset pipeline, import asset.

Aklında kalsın: Kod olmayan her şey asset değildir; bazı Resource dosyaları da asset kabul edilir.

API TEMEL

Bir sistemin koddan kullanmana izin verdiği sınıf, metod, property ve kurallar bütünüdür.

Nerede duyarsın? Godot API, Input API, Steam API.

Aklında kalsın: API = "bu sistemle kod üzerinden nasıl konuşurum?"

Library / Framework / Plugin TEMEL

Library belirli işlevler sunan kod paketi; framework uygulamayı belli bir yapıda kurdurur; plugin/add-on ise var olan araca özellik ekler.

Nerede duyarsın? third-party library, Godot plugin.

Aklında kalsın: Üçü benzer görünür ama kapsamaları farklıdır.

Build / Export TEMEL

Projenin oyuncunun çalıştırabileceği platform çıktısına dönüştürülmesidir. Godot genellikle "Export" kelimesini kullanır.

Nerede duyarsın? Windows build, export preset, debug build.

Aklında kalsın: Editor'de çalışan proje ile dağıttığın oyun dosyası aynı şey değildir.

Runtime TEMEL

Oyunun gerçekten çalıştığı zaman dilimi ve çalışma ortamıdır.

Nerede duyarsın? runtime error, runtime instance.

Aklında kalsın: Editor zamanı ile oyun çalışırken olanlar farklı olabilir.

Hızlı sözlük

Tool / Tooling	Üretimi kolaylaştıran editör, profiler, importer, özel araç ve otomasyonların genel adı.
SDK	Belirli bir platform veya servis için geliştirici araçları ve API paketleri.
IDE / Code Editor	Kod yazdığın araç. Godot script editor, VS Code, Rider gibi.
Import	Dışarıdan gelen asseti motorun kullanacağı biçime hazırlama süreci.
Pipeline	Bir içeriğin üretimden oyuna girene kadar geçtiği adımlar zinciri.
Dependency	Bir sistemin çalışmak için ihtiyaç duyduğu başka paket, sınıf veya servis.
Open Source	Kaynak kodu erişilebilir ve lisansı izin verdiği ölçüde incelenebilir/değiştirilebilir yazılım.
License	Bir yazılımın veya assetin hangi koşullarla kullanılacağını belirleyen hukuki izin.
Repository (Repo)	Proje dosyalarının version control altında tutulduğu depo.
Platform	Windows, Linux, Android, iOS, Web, konsol gibi hedef çalışma ortamı.

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlele söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. Engine ile editor arasındaki fark nedir?
2. API kelimesini kendi cümlele nasıl açıklarsın?
3. Export neden "Save" ile aynı şey değildir?

BÖLÜM 02

Programlama ve GDScript temelleri

Kod konuşmalarında en sık geçen dil terimleri.

HEDEF Bu bölüm bittiğinde 12 ana terimi açıklayabilecek ve 21 ek terimi duyduğunda tanıyabileceksin.

Variable (Değişken) TEMEL

Programın çalışma sırasında tuttuğu isimlendirilmiş veridir. Değeri değişebilir.

Nerede duyarsın? "speed variable", "local variable".

Aklında kalsın: Değişken = etiketlenmiş veri kutusu.

```
var health: int = 100
```

Constant (Sabit) TEMEL

Program çalışırken değiştirilmemesi amaçlanan değerdir. GDScript'te const ile tanımlanır.

Nerede duyarsın? constants, magic numbers.

Aklında kalsın: Değişmeyecek bir değere anlamlı isim ver.

```
const MAX_HEALTH := 100
```

Data Type (Veri Tipi) TEMEL

Bir değer ne tür veri olduğunu belirtir: int, float, bool, String, Vector2 gibi.

Nerede duyarsın? type mismatch, typed variable.

Aklında kalsın: Tip, o veriye hangi işlemlerin uygulanabileceğini de belirler.

Dynamic vs Static Typing TEMEL

Dynamic kullanımda tip çalışma anında belirlenebilir. Static typing'de değişken/parametre/return tipi açıkça belirtilir. GDScript ikisini de destekler.

Nerede duyarsın? typed GDScript, type hint.

Aklında kalsın: Yeni başlarken tip belirtmek hata yakalamayı ve autocomplete'i kolaylaştırır.

```
var speed: float = 220.0
func damage(amount: int) -> void:
    health -= amount
```

Function / Method TEMEL

Belirli işi yapan tekrar kullanılabilir kod bloğudur. Bir nesne/sınıf bağlamındaki fonksiyona genellikle method denir.

Nerede duyarsın? call a function, override a method.

Aklında kalsın: İyi fonksiyonun görevi nettir.

```
func heal(amount: int) -> void:
    health += amount
```

Parameter vs Argument TEMEL

Parameter fonksiyon tanımındaki isimdir; argument fonksiyonu çağırırken verdiğin gerçek değerdir.

Nerede duyarsın? pass an argument.

Aklında kalsın: func heal(amount) içindeki amount parameter; heal(25) içindeki 25 argument.

Return Value TEMEL

Bir fonksiyonun çağırana geri verdiği sonuçtur.

Nerede duyarsın? return type, return value.

Aklında kalsın: Fonksiyon yalnızca iş yapmak zorunda değildir; sonuç da üretebilir.

```
func get_damage() -> int:
    return strength * 2
```

Scope (Kapsam) TEMEL

Bir değişkenin kodun hangi bölümünden erişilebilir olduğunu belirler: local, sınıf/script kapsamı vb.

Nerede duyarsın? out of scope, local variable.

Aklında kalsın: Her şeyi global yapmak kolay görünür ama projeyi karıştırır.

Conditional TEMEL

Koşula göre farklı kod yollarını çalıştırır. if / elif / else ve match bu amaçla kullanılır.

Nerede duyarsın? condition, branch.

Aklında kalsın: Oyun mantığının büyük bölümü "eğer X ise Y yap"tır.

```
if health <= 0:
    die()
```

Loop (Döngü) TEMEL

Aynı işlemi bir koleksiyon veya koşul üzerinden tekrarlar. for ve while temel döngülerdir.

Nerede duyarsın? iterate, loop through array.

Aklında kalsın: Sonsuz while döngülerine dikkat et.

```
for player in players:
    print(player.name)
```

Array / Dictionary TEMEL

Array sıralı değer listesi; Dictionary anahtar-değer eşleşmesi tutar.

Nerede duyarsın? list of enemies, stats dictionary.

Aklında kalsın: Array'da index; Dictionary'de key düşün.

```
var players: Array[String] = ["Ali", "Ece"]
var stats := {"speed": 80, "shot": 72}
```

Class / Object / Instance TEMEL

Class bir tür/şablon fikridir; object o türden çalışan nesnedir; instance ise bir sınıfın veya sahnenin oluşturulmuş örneğidir.

Nerede duyarsın? instantiate, object instance.

Aklında kalsın: Player sınıf fikri; sahnedeki Player_1 çalışan örnektir.

Hızlı sözlük

int	Tam sayı: 5, -12, 100.
float	Ondalıklı sayı: 3.5, 0.016.
bool	true / false.

String	Metin verisi.
Variant	Godot'un pek çok farklı veri tipini taşıyabilen genel değer türü.
Operator	+, -, *, /, ==, !=, >, && gibi işlem işaretleri.
Expression	Bir değere hesaplanan kod parçası: damage * 2 + bonus.
Statement	Programın yürüttüğü tam talimat satırı/yapısı.
Enum	İsmlendirilmiş sabit seçenek kümesi: IDLE, RUNNING, DEAD.
null	Geçerli bir nesne/değer referansı olmadığını ifade eder.
Casting	Bir değeri başka bir tipe dönüştürme veya o tipte ele alma.
Annotation	GDScript'te @ ile başlayan özel işaretler: @export, @onready.
@export	Bir property'yi Inspector'da düzenlenebilir hale getiren annotation.
@onready	Node hazır olduğunda bir değişkeni initialize etmeyi kolaylaştıran annotation.
extends	Bir scriptin hangi Godot sınıfından veya custom class'tan türediğini belirtir.
class_name	Script sınıfına global olarak kullanılabilen isim verir.
Inheritance	Bir sınıfın üst sınıftan davranış/özellik alması.
Composition	Davranışları tek dev sınıf yerine küçük parçaları birleştirerek kurma yaklaşımı.
Callable	Sonradan çağrılacak bir fonksiyonu temsil eden Godot türü.
Lambda	İsimsiz/yerinde tanımlanan küçük fonksiyon.
await	Bir signal veya coroutine benzeri işlemin tamamlanmasını bekleyip sonra devam etme.

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlelerle söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. Parametre ve argument farkını örnekle açıkla.
2. Array ile Dictionary arasında temel fark nedir?
3. Static typing sana başlangıçta hangi iki faydayı sağlar?

BÖLÜM 03

Godot'un kalbi: Node, Scene ve SceneTree

Godot kurslarında sürekli duyacağın yapısal kavramlar.

HEDEF Bu bölüm bittiğinde 10 ana terimi açıklayabilecek ve 10 ek terimi duyduğunda tanıyabileceksin.

Node TEMEL

Godot'un temel yapı taşıdır. Sprite göstermek, ses çalmak, fizik gövdesi olmak veya UI elemanı olmak gibi belirli sorumluluklar taşır.

Nerede duyarsın? "add a node", "node type".

Aklında kalsın: Godot'ta oyunu küçük node parçalarından kurarsın.

Scene TEMEL

Bir ağaç halinde organize edilmiş node grubudur ve tekrar kullanılabilir. Kaydedildiğinde .tscn gibi bir scene dosyasına dönüşebilir.

Nerede duyarsın? player scene, main scene.

Aklında kalsın: Scene yalnızca "level" değildir; Player, mermi, menü de scene olabilir.

SceneTree TEMEL

Oyun çalışırken aktif node ağacını yöneten ana yapıdır.

Nerede duyarsın? get_tree(), scene tree.

Aklında kalsın: Sahnedeki hiyerarşinin runtime karşılığı.

Parent / Child TEMEL

SceneTree hiyerarşisindeki üst-alt node ilişkisidir. Child genellikle parent transformundan/hayat döngüsünden etkilenir.

Nerede duyarsın? child node, reparent.

Aklında kalsın: Hiyerarşi sadece görüntü değil, davranışı da etkileyebilir.

NodePath / get_node TEMEL

Bir node'a ağaç içindeki yolu üzerinden ulaşma yöntemidir. \$HealthBar kısa sözdizimlerinden biridir.

Nerede duyarsın? node path, get node reference.

Aklında kalsın: Çok kırılğan uzun yollar yerine temiz scene yapısı ve export referansları tercih edilebilir.

```
@onready var bar: ProgressBar = $UI/HealthBar
```

PackedScene / instantiate() TEMEL

Kaydedilmiş bir scene kaynağını temsil eder. instantiate() ile runtime'da yeni bir scene örneği üretirsin.

Nerede duyarsın? spawn bullet, instantiate enemy.

Aklında kalsın: "Scene dosyası" kalıp; instantiate edilmiş node ağacı çalışan örnektir.

```
var enemy := enemy_scene.instantiate()
```



```
add_child(enemy)
```

_ready() TEMEL

Node SceneTree'ye girip çocukları hazır olduğunda çağrılan temel lifecycle callback'lerinden biridir.

Nerede duyarsın? ready function.

Aklında kalsın: Node referanslarını ve başlangıç kurulumunu burada sık görürsün.

```
func _ready() -> void:
    print("Hazır")
```

queue_free() TEMEL

Bir Node'u güvenli biçimde silinmek üzere kuyruğa alır.

Nerede duyarsın? despawn, free the node.

Aklında kalsın: "Öldü" diye görünmez yapmak ile gerçekten node'u kaldırmak farklıdır.

```
func die() -> void:
    queue_free()
```

Group TEMEL

Node'ları hiyerarşiden bağımsız etiketleyip topluca bulmak/işlemek için kullanılan sistemdir.

Nerede duyarsın? enemy group, get_nodes_in_group.

Aklında kalsın: "Bu node Enemy mi?" gibi kategorik işlemler için kullanışlıdır.

Autoload TEMEL

Bir scene veya scripti proje boyunca otomatik yüklenen global erişilebilir node olarak kullanma mekanizmasıdır.

Nerede duyarsın? GameManager autoload, global singleton.

Aklında kalsın: Her şeyi Autoload yapmak yerine gerçekten global ömürlü servisler için kullan.

Hızlı sözlük

Root Node	Bir scene'in en üst node'u.
Main Scene	Proje çalıştırıldığında başlangıç olarak açılan scene.
Owner	Bir node'un hangi scene kaynağına ait olarak kaydedileceğini etkileyen property.
add_child()	Runtime'da bir Node'u SceneTree'ye child olarak ekler.
remove_child()	Node'u parent'tan ayırır; tek başına silmez.
reparent()	Node'un parent'ını değiştirme işlemi.
Unique Node	Scene içinde %Name benzeri benzersiz ad erişimi için işaretlenen node.
Lifecycle	Bir nesnenin oluşturulma, hazır olma, güncellenme ve yok edilme aşamaları.
Notification	Godot Object/Node yaşam döngüsündeki çeşitli olayların düşük seviye bildirim sistemi.
Signal	Bir Object'in "bir şey oldu" diye dinleyicilere mesaj yayınlama mekanizması.

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlele söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. Scene yalnızca level anlamına mı gelir?
2. instantiate() ne yapar?
3. Autoload için iyi bir kullanım örneği ver.

BÖLÜM 04

Game loop, frame, delta ve input

"Neden delta ile çarpıyoruz?" sorusunun oturduğu bölüm.

HEDEF Bu bölüm bittiğinde 8 ana terimi açıklayabilecek ve 10 ek terimi duyduğunda tanıyabileceksin.

Game Loop **TEMEL**

Oyun çalışırken input alma, dünya durumunu güncelleme, fizik ve çizim gibi adımların sürekli tekrarlanmasıdır.

Nerede duyarsın? update loop, main loop.

Aklında kalsın: Oyun tek sefer çalışan program değil, sürekli güncellenen sistemdir.

Frame / FPS **TEMEL**

Frame ekrana üretilen bir görüntü karesidir. FPS saniyedeki frame sayısıdır.

Nerede duyarsın? 60 FPS, frame rate.

Aklında kalsın: FPS görüntü sıklığıdır; oyun mantığını doğrudan FPS'ye bağlama.

_process(delta) **TEMEL**

Her çizim frame'inde çalışabilen callback'tir. UI, görsel takip, frame-temelli işler için sık kullanılır.

Nerede duyarsın? process callback.

Aklında kalsın: Fizik hareketini genellikle _physics_process'te tut.

```
func _process(delta: float) -> void:
    rotation += 1.5 * delta
```

_physics_process(delta) **TEMEL**

Sabit fizik adımlarına yakın aralıklarla çalışan callback'tir. CharacterBody hareketi ve fizik ilişkili hesaplar burada yapılır.

Nerede duyarsın? physics tick.

Aklında kalsın: Fizik için daha tutarlı zaman adımı sağlar.

delta **TEMEL**

Son güncelleme adımından beri geçen süreyi saniye cinsinden temsil eden değerdir.

Nerede duyarsın? multiply by delta.

Aklında kalsın: "Saniyede 200 piksel" gibi zamana bağlı hızları frame bağımsız yapmak için kullanılır.

```
position.x += speed * delta
```

Input Action / Input Map **TEMEL**

Fiziksel tuşları doğrudan kodlamak yerine "move_left", "jump" gibi eylemler tanımlarsın; bunları Project Settings > Input Map'te cihaz girdilerine bağlarsın.

Nerede duyarsın? action pressed, input map.

Aklında kalsın: Kod "Space" değil "jump" eylemini bilir.

```
if Input.is_action_just_pressed("jump"):
    jump()
```

Polling vs Event-driven Input TEMEL

Polling o anki input durumunu sürekli sorar; event-driven yaklaşım gelen InputEvent'i olay olarak işler.

Nerede duyarsın? `_input(event)`, `Input.is_action_pressed`.

Aklında kalsın: Sürekli hareket polling; tekil UI/tuş olayları event yaklaşımına uygun olabilir.

Timer / Cooldown TEMEL

Timer belirli süre sonra veya aralıklı olay üretir. Cooldown ise bir aksiyonun tekrar kullanılmadan önce bekleme süresidir.

Nerede duyarsın? `attack cooldown`, `Timer node`.

Aklında kalsın: Cooldown bir tasarım kavramı; Timer bunu uygulamanın araçlarından biridir.

Hızlı sözlük

Timestep	Simülasyonun her güncellemede ilerlettiği zaman miktarı.
Fixed Timestep	Özellikle fizik için sabit aralıklarla güncelleme yaklaşımı.
Physics Tick	Bir fizik güncelleme adımı.
VSync	Frame üretimini ekran yenileme döngüsüyle senkronlamayı amaçlayan seçenek.
InputEvent	Klavye, mouse, gamepad vb. bir giriş olayını taşıyan Godot veri yapısı.
Pressed / Just Pressed	Basılı tutuluyor mu / bu frame'de yeni mi basıldı ayrımı.
Dead Zone	Analog stick küçük titreşimlerinin input sayılmaması için merkez çevresindeki tolerans.
Focus	UI'da hangi Control'un klavye/gamepad inputunu alacağını belirleyen durum.
Pause	SceneTree veya node processing davranışının oyun duraklatıldığında değişmesi.
Time Scale	Oyunun zaman akışını hızlandırmak/yavaşlatmak için kullanılan ölçek fikri.

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlele söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. delta neyi temsil eder?
2. Neden Input Map kullanmak iyi fikirdir?
3. `_process` ve `_physics_process` için basit kullanım ayrımı yap.

BÖLÜM 05

Koordinatlar, Vector ve oyun matematiği

Oyun geliştirmenin “matematik korkutmayan” temel sözlüğü.

HEDEF Bu bölüm bittiğinde 8 ana terimi açıklayabilecek ve 12 ek terimi duyduğunda tanıyabileceksin.

Coordinate System TEMEL

Dünyadaki konumu eksenlerle ifade eden sistemdir. 2D’de x/y, 3D’de x/y/z kullanılır.

Nerede duyarsın? coordinates, axis.

Aklında kalsın: Ekran koordinatları ile dünya koordinatları aynı olmak zorunda değildir.

Vector2 / Vector3 TEMEL

Yön, konum farkı, hız gibi birden fazla sayıyı birlikte temsil eden temel matematik türleridir.

Nerede duyarsın? direction vector, velocity vector.

Aklında kalsın: Vector sadece “konum” değildir.

```
var direction := Vector2(1, 0)
var velocity := direction * speed
```

Magnitude / Length TEMEL

Bir vector’ün büyüklüğüdür. Örneğin hız vector’ünün length’i gerçek hız miktarını verebilir.

Nerede duyarsın? vector length, magnitude.

Aklında kalsın: Yön + büyüklük ayrımını anlamak vectorleri kolaylaştırır.

Normalize TEMEL

Vector’ün yönünü koruyup uzunluğunu 1 yapmaktır.

Nerede duyarsın? normalized direction.

Aklında kalsın: Diagonal hareketin daha hızlı olmasını engellemekte sık kullanılır.

```
direction = direction.normalized()
```

Local vs Global Coordinates TEMEL

Local koordinat parent’a göre; global koordinat scene/world referansına göre konumu ifade eder.

Nerede duyarsın? global_position, local transform.

Aklında kalsın: Parent hareket edince child’ın global konumu değişebilir.

Transform TEMEL

Position, rotation ve scale gibi uzaysal dönüşümlerin birlikte temsilidir.

Nerede duyarsın? transform, global_transform.

Aklında kalsın: Nesnenin “dünyada nasıl yerleştiği”nin paketlenmiş hali.

Interpolation / Lerp TEMEL

İki değer arasında yumuşak geçiş üretme yaklaşımıdır. Lerp lineer interpolasyonun yaygın biçimidir.

Nerede duyarsın? lerp camera, smoothing.

Aklında kalsın: Bir anda sıçramak yerine hedefe yaklaşmak için kullanılır.

Distance / Direction TEMEL

İki nokta arasındaki mesafe ve birinden diğerine yön oyun mantığında çok sık hesaplanır.

Nerede duyarsın? distance_to, direction_to.

Aklında kalsın: AI takip, saldırı menzili, hedefleme gibi sistemlerin temeli.

Hızlı sözlük

Origin	Koordinat sisteminin başlangıç noktası.
Axis	x, y, z yönlerinden biri.
Position	Nesnenin konumu.
Rotation	Nesnenin yönelimi/dönüş açısı.
Scale	Nesnenin boyut ölçeği.
Angle	İki yön arasındaki açısal değer.
Degree / Radian	Açıyı ifade eden iki birim. Programlama API'lerinde radyan sık kullanılır.
Dot Product	İki vector'ün yön benzerliğini ölçmede kullanılan işlem; görüş konisi vb. ileri konularda çıkar.
Cross Product	3D'de vectorlere dik yön üretme gibi işlemlerde kullanılan çarpım.
Clamp	Bir değeri minimum ve maksimum sınır arasında tutma.
Remap	Bir değer aralığını başka aralığa dönüştürme.
Smoothing	Ani değişimleri daha yumuşak hale getirme genel yaklaşımı.

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlele söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. Normalize edilmiş vector'ün length'i yaklaşık kaçtır?
2. Local position ile global position farkı nedir?
3. Lerp nerede işe yarar?

BÖLÜM 06

Physics, collision ve hareket

CharacterBody2D gördüğünde ne ailesine ait olduğunu bil.

HEDEF Bu bölüm bittiğinde 9 ana terimi açıklayabilecek ve 11 ek terimi duyduğunda tanıyabileceksin.

Collision TEMEL

İki fizik şeklinin birbirine temas/örtüşme durumunun motor tarafından algılanmasıdır.

Nerede duyarsın? collision detection.

Aklında kalsın: Görsel sprite ile collision shape aynı şey değildir.

CollisionShape2D / 3D TEMEL

Bir fizik nesnesinin çarpışma geometrisini tanımlar. Rectangle, capsule, sphere vb. şekiller kullanılabilir.

Nerede duyarsın? hitbox shape, collision shape.

Aklında kalsın: Görsel mümkün olduğunca uygun ama gereksiz karmaşık olmayan şekil kullan.

CharacterBody TEMEL

Kodla kontrol edilen oyuncu/NPC karakterleri gibi hareketli gövdeler için tasarlanmış body türüdür.

Nerede duyarsın? move_and_slide, CharacterBody2D.

Aklında kalsın: RigidBody gibi tamamen fizik simülasyonuna bırakılmaz.

RigidBody TEMEL

Hareketi fizik motorunun kuvvet, impulse, gravity gibi etkilerle simüle ettiği body türüdür.

Nerede duyarsın? rigid body physics.

Aklında kalsın: Kutu, top, fizik objesi gibi "simülasyona bırakılan" nesneler.

StaticBody TEMEL

Hareket etmeyen çevre çarpışmaları için kullanılan body türüdür.

Nerede duyarsın? wall collision, static body.

Aklında kalsın: Duvar/zemin gibi sabit geometri.

Area TEMEL

Fiziksel olarak duvar oluşturmak yerine başka body/area giriş-çıkışlarını algılayan bölgedir.

Nerede duyarsın? trigger zone, Area2D.

Aklında kalsın: Pickup, damage zone, checkpoint için çok kullanılır.

Velocity / Acceleration TEMEL

Velocity hız ve yönü; acceleration velocity'nin zamanla değişimini ifade eder.

Nerede duyarsın? velocity vector, acceleration.

Aklında kalsın: position doğrudan değiştirmek ile velocity tabanlı hareket farklıdır.

Collision Layer / Mask TEMEL

Layer “ben hangi fizik katmanındayım?”, mask ise “hangi katmanları algılayırım?” mantığıdır.

Nerede duyarsın? layer 1, mask 2.

Aklında kalsın: Çarpışma filtrelemenin temelidir.

RayCast / ShapeCast TEMEL

Belirli yön/şekil boyunca çarpışma sorgusu yapar. Görüş, zemin kontrolü, silah isabeti gibi işlerde kullanılır.

Nerede duyarsın? raycast hit, ground check.

Aklında kalsın: Ray noktasal çizgi; ShapeCast hacimli şekil süpürmesi gibi düşün.

Hızlı sözlük

Gravity	Nesneleri belirli yönde hızlandıran yerçekimi etkisi.
Force	Bir rigid body'nin hareketini zaman içinde değiştiren kuvvet.
Impulse	Kısa süreli ani momentum değişimi; “tekme” gibi.
Friction	Temas eden yüzeylerin kaymaya karşı direnci.
Bounce / Restitution	Çarpışmada ne kadar sekme olacağını belirleyen fizik özelliği.
Normal	Bir yüzeye dik yön; çarpışma yüzeyinin yönünü anlamada kullanılır.
Kinematic	Genel oyun geliştirme dilinde kodla belirlenen hareket yaklaşımı; Godot 4'te karakter türü CharacterBody adını kullanır.
Trigger	Başka collider'ı engellemeden giriş/çıkış olayı üreten bölge için genel terim; Godot'ta Area sık karşılığıdır.
Hitbox	Genellikle saldırının vurduğu alan.
Hurtbox	Genellikle hasar alabilen hedef alanı.
Physics Material	Friction ve bounce gibi fizik yüzey davranışlarını tanımlayan kaynak.

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlelerle söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. Area ile StaticBody arasındaki temel fark nedir?
2. Layer ve mask'i tek cümlede ayır.
3. RayCast için iki kullanım örneği ver.

BÖLÜM 07

Görsel, 2D/3D, kamera ve animasyon

Renderer konuşmalarında duyacağın kelimeler.

HEDEF Bu bölüm bittiğinde 9 ana terimi açıklayabilecek ve 14 ek terimi duyduğunda tanıyabileceksin.

Sprite / Texture TEMEL

Texture görüntü verisidir; Sprite2D bu texture'ı 2D dünyada gösteren node türlerinden biridir.

Nerede duyarsın? sprite sheet, texture filter.

Aklında kalsın: Dosya = texture; sahnedeki gösteren nesne = sprite.

Atlas / Sprite Sheet TEMEL

Birden fazla küçük görseli tek büyük texture içinde saklama yaklaşımıdır.

Nerede duyarsın? sprite atlas, animation frames.

Aklında kalsın: Animasyon frame'leri ve UI ikonlarında sık görürsün.

Tile / TileMap TEMEL

Tile tekrar kullanılabilir parça; TileMap bu parçalarla grid tabanlı 2D dünya oluşturmaya yarayan sistemdir.

Nerede duyarsın? tileset, tilemap layer.

Aklında kalsın: Her duvarı ayrı Sprite yapmak yerine tile tabanlı üretim.

Camera TEMEL

Oyuncuya dünyanın hangi bölümünün ve nasıl gösterileceğini belirler.

Nerede duyarsın? Camera2D smoothing, FOV.

Aklında kalsın: Kamera sadece takip etmez; zoom, shake, framing ve game feel'in parçasıdır.

Viewport TEMEL

Bir sahnenin/görüntünün render edildiği yüzey/alan. Ana pencere de bir viewport mantığı taşır; ayrıca alt viewport'lar yapılabilir.

Nerede duyarsın? SubViewport, viewport texture.

Aklında kalsın: Mini-map, render-to-texture gibi ileri kullanımlarda çıkar.

Material / Shader TEMEL

Material bir yüzeyin nasıl çizileceğine ilişkin ayar/kaynak; shader GPU üzerinde görüntüyü nasıl hesaplayacağını belirleyen programdır.

Nerede duyarsın? shader code, material property.

Aklında kalsın: Shader = çizim mantığı; material = o mantığın belirli ayarlarla kullanımı.

AnimationPlayer TEMEL

Property değerlerini zaman içinde keyframe'lerle değiştiren genel animasyon aracıdır.

Nerede duyarsın? play animation, animation track.

Aklında kalsın: Karakter dışında UI, kapı, kamera, renk gibi her şeyi animasyonlayabilir.

Tween **TEMEL**

Bir değeri başlangıçtan hedefe belirli süre/easing ile geçirmek için kullanılan hafif animasyon yaklaşımıdır.

Nerede duyarsın? tween position, easing.

Aklında kalsın: Basit UI ve hareket geçişleri için çok pratiktir.

Draw Call **TEMEL**

CPU'nun GPU'ya "şu geometriyi/öğeyi çiz" şeklinde gönderdiği render komutlarının genel terimidir.

Nerede duyarsın? reduce draw calls, batching.

Aklında kalsın: Çok sayıda draw call performans darboğazı olabilir; ölçmeden optimize etme.

Hızlı sözlük

Renderer	Sahneyi ekrandaki piksellere dönüştüren grafik sistemi.
Canvas	Godot 2D çizim dünyasının temel render katmanı kavramı.
Z-index	2D öğelerin önde/arkada çizim sırasını etkileyen değer.
Layer	Görsel veya mantıksal öğeleri ayırmak için kullanılan katman kavramı; collision layer ile karıştırma.
Pixel Art	Piksel ölçüsünün bilinçli tasarım unsuru olduğu düşük çözünürlüklü görsel stil.
Filtering	Texture büyütülür/küçültülürken piksellerin nasıl örnekleneyeceği.
Mipmaps	Texture'ın uzak/küçük görünüşleri için önceden oluşturulan daha düşük çözünürlüklü seviyeler.
Particle	Çok sayıda küçük öğeyle duman, ateş, kıvılcım vb. efekt üretme sistemi.
Light	Sahne aydınlatma kaynağı.
Keyframe	Animasyonda belirli zamanda belirli değeri işaretleyen ana kare.
Easing	Animasyon hızının başta/ortada/sonda nasıl değişeceğini tanımlayan eğri/karakter.
AnimationTree	Daha karmaşık animasyon blend ve state geçişlerini yöneten Godot sistemi.
Blend	İki animasyon veya değeri oranlı şekilde karıştırma.
FOV	3D kameranin görüş açısı (field of view).

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlele söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. Texture ile Sprite arasındaki fark nedir?
2. Shader ve material arasındaki ilişkiyi açıkla.
3. Tween ne zaman AnimationPlayer'dan daha pratik olabilir?

BÖLÜM 08

UI, resolution ve responsive tasarım

Menü ve simülasyon oyunlarında özellikle kritik.

HEDEF Bu bölüm bittiğinde 9 ana terimi açıklayabilecek ve 12 ek terimi duyduğunda tanıyabileceksin.

UI / GUI TEMEL

User Interface, oyuncunun bilgi gördüğü ve etkileştiği arayüzdür: buton, panel, menü, HUD vb. GUI çoğu bağlamda grafik arayüz anlamında kullanılır.

Nerede duyarsın? UI system, HUD, menu.

Aklında kalsın: UI sadece "güzel görünüm" değil, etkileşim ve bilgi mimarisidir.

Control TEMEL

Godot'ta UI node'larının temel sınıf ailesidir. Position/size yerine anchors, offsets ve containers gibi UI düzen mantıklarıyla çalışır.

Nerede duyarsın? Control node, custom minimum size.

Aklında kalsın: Node2D ile UI yapmaya çalışmak yerine Control ailesini öğren.

Container TEMEL

Child Control node'larının boyut/konumunu otomatik düzenleyen UI node'larıdır. VBox, HBox, Grid, Margin gibi çeşitleri vardır.

Nerede duyarsın? VBoxContainer, layout.

Aklında kalsın: Responsive UI'nın en güçlü araçlarından biridir.

Anchor / Offset TEMEL

Anchor parent boyutuna göre oransal referans; offset ise bu referansa eklenen piksel uzaklığıdır.

Nerede duyarsın? anchors preset, offsets.

Aklında kalsın: Farklı çözünürlükte UI'nın yerini korumasına yardım eder.

Resolution TEMEL

Ekranın piksel boyutudur: 1920x1080 gibi.

Nerede duyarsın? base resolution, window size.

Aklında kalsın: Resolution ile aspect ratio farklı kavramlardır.

Aspect Ratio TEMEL

Ekranın genişlik/yükseklik oranıdır: 16:9, 21:9 gibi.

Nerede duyarsın? ultrawide support, aspect ratio.

Aklında kalsın: Aynı aspect ratio farklı resolution'larda olabilir.

Scaling / Stretch TEMEL

Oyunun veya UI'nın farklı pencere boyutlarına nasıl ölçekleneceğini belirleyen yaklaşım/ayarlardır.

Nerede duyarsın? stretch mode, scaling.
Aklında kalsın: Responsive davranışı test etmek şarttır.

Theme TEMEL

UI öğelerinin font, renk, stylebox, ikon vb. görünüm kurallarını merkezi biçimde tanımlayan sistemdir.

Nerede duyarsın? Theme resource, theme override.

Aklında kalsın: Her butonu tek tek biçimlendirmek yerine ortak tema.

HUD TEMEL

Oyun sırasında sürekli veya sık görülen bilgi arayüzü: can, skor, minimap, ammo vb.

Nerede duyarsın? Heads-Up Display.

Aklında kalsın: Main menu'den farklı, oyun içi arayüz katmanıdır.

Hızlı sözlük

Responsive UI	Farklı ekran ve pencere boyutlarına uyum sağlayan arayüz.
Layout	UI öğelerinin yerleşim düzeni.
Margin / Padding	Öğeler arasındaki veya içerik ile kenar arasındaki boşluk kavramları.
StyleBox	Godot Theme sisteminde panel/buton arka planı, border ve radius gibi stil kaynağı.
Minimum Size	Bir Control'un küçülmeyeceği tercih edilen alt boyut.
Focus Navigation	Klavye/gamepad ile UI elemanları arasında odak geçişi.
Localization (L10n)	Oyunun metin/içeriğini farklı dil ve bölgelere uyarlama.
Internationalization (i18n)	Yazılımı farklı dil/bölgelere uygun tasarlama süreci.
DPI	Fiziksel ekran yoğunluğuyla ilişkili ölçü; özellikle mobil ve yüksek yoğunluklu ekranlarda UI ölçekleme konuşmalarında çıkar.
Safe Area	Çentik, rounded corner vb. nedeniyle UI'nın güvenli yerleşebileceği ekran alanı.
Tooltip	Hover/focus ile gösterilen yardımcı açıklama.
Modal	Açıkken arka plan etkileşimini kısıtlayan öncelikli pencere/dialog.

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlele söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. Resolution ile aspect ratio farkını açıkla.
2. Container kullanmanın ana avantajı nedir?
3. Theme neden büyük UI projelerinde önemlidir?

BÖLÜM 09

Data, Resource, save/load ve mimari

Küçük projeyi büyük projeden ayıran düşünme biçimleri.

HEDEF Bu bölüm bittiğinde 10 ana terimi açıklayabilecek ve 13 ek terimi duyduğunda tanıyabileceksin.

Data vs Logic **TEMEL**

Data "ne var?" bilgisidir; logic "bu bilgiyle ne yapılır?" davranışdır. Oyuncu speed=80 data; calculate_speed() logic örneğidir.

Nerede duyarsın? separate data from logic.

Aklında kalsın: Veri yoğun oyunlarda bu ayrım projeyi çok rahatlatır.

Resource **TEMEL**

Godot'un veri ve asset taşıyan temel nesne türlerinden biridir. Custom Resource ile item, karakter, skill, kart vb. veri şablonları oluşturabilirsin.

Nerede duyarsın? Resource file, .tres, custom resource.

Aklında kalsın: Scene daha çok node ağacı; Resource daha çok paylaşılabılır veri/asset.

Serialization **TEMEL**

Runtime verisini dosyaya yazılabilecek/taşınabilecek biçime dönüştürme ve geri okuma sürecidir.

Nerede duyarsın? serialize save data.

Aklında kalsın: Save sistemi yalnızca "dosyaya yaz" değil, veriyi temsil etme problemidir.

Save / Load **TEMEL**

Oyunun kalıcı durumunu saklama ve daha sonra geri yükleme sistemidir.

Nerede duyarsın? save game, load profile.

Aklında kalsın: Node'un tamamını körlemesine kaydetmek yerine gerekli veriyi seç.

State **TEMEL**

Bir nesnenin belirli andaki durumunu ifade eder: IDLE, RUNNING, ATTACKING, DEAD gibi.

Nerede duyarsın? player state, game state.

Aklında kalsın: State hem küçük bir karakter durumu hem oyunun genel durumu olabilir.

Finite State Machine (FSM) **TEMEL**

Bir sistemi sınırlı state'ler ve aralarındaki geçişlerle modelleyen yaklaşımdır.

Nerede duyarsın? state machine, transition.

Aklında kalsın: AI ve karakter davranışlarında çok sık duyarsın.

Coupling **TEMEL**

İki sistemin birbirine ne kadar sıkı bağımlı olduğunu anlatır.

Nerede duyarsın? tightly coupled, decoupled.

Aklında kalsın: Düşük coupling genellikle sistemi değiştirmeyi/test etmeyi kolaylaştırır.

Cohesion **TEMEL**

Bir sınıf veya modüldeki parçaların tek ve anlamlı bir sorumluluk etrafında ne kadar iyi toplandığını anlatır.

Nerede duyarsın? high cohesion.

Aklında kalsın: Bir script her işi yapıyorsa cohesion genellikle düşüktür.

Encapsulation **TEMEL**

Bir sistemin iç detaylarını saklayıp dışarıya kontrollü bir arayüz sunma fikridir.

Nerede duyarsın? private implementation, public API.

Aklında kalsın: Başka scriptler her değişkenini kurcalamak zorunda kalmamalı.

Signal / Event-driven Architecture **TEMEL**

Bir sistemin diğerinin içini doğrudan bilmek yerine olay yayınlayıp dinleyicilerin tepki vermesi yaklaşımıdır.

Nerede duyarsın? event bus, signal connection.

Aklında kalsın: Player "UI güncelle" demek yerine health_changed yayınlabilir.

```
signal health_changed(value: int)

func take_damage(amount: int) -> void:
    health -= amount
    health_changed.emit(health)
```

Hızlı sözlük

JSON	İnsan tarafından okunabilir yaygın veri değişim metin formatı.
Config	Ayar verilerini temsil eden genel kavram.
Schema	Bir veri yapısında hangi alanların/tiplerin bulunacağını tanımlayan yapı.
Manager	Belirli bir sistemin koordinasyonundan sorumlu sınıf/node için yaygın isim; gereğinden fazla manager üretmemeye dikkat.
Singleton	Uygulama bağlamında tek erişim noktası/tek örnek olarak tasarlanan nesne paterni; Godot Autoload ile sık ilişkilendirilir ama aynı kavram değildir.
Event Bus	Birçok sistemin ortak olay kanalı üzerinden haberleşmesi yaklaşımı.
Abstraction	Gereksiz iç ayrıntıları saklayıp önemli davranışı daha basit arayüzle sunma.
Interface	Bir nesnenin hangi davranışları sağlayacağını tanımlayan sözleşme fikri. GDScript'te klasik interface anahtar kelimesi yoktur; desen olarak uygulanabilir.
Dependency Injection	Bir nesnenin ihtiyacını kendisi yaratmak yerine dışarıdan alması yaklaşımı.
Composition over Inheritance	Davranışları derin inheritance zincirleri yerine küçük parçaları birleştirerek kurmayı tercih eden tasarım ilkesi.
Object Pooling	Sık oluşturulup silinen nesneleri yeniden kullanmak için havuzda tutma tekniği.
Factory	Nesne oluşturma kararlarını tek yerde toplayan tasarım yaklaşımı.
Module	Belirli bir özelliği/sorumluluğu kapsayan bağımsız kod bölümü.

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlele söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. Resource ile Scene için basit ayrım yap.
2. Coupling neden önemlidir?
3. State machine hangi probleme çözüm getirir?

BÖLÜM 10

Gameplay, AI ve sistem tasarımı terimleri

Bir oyun geliştirici sohbetinde çok sık geçen davranış kelimeleri.

HEDEF Bu bölüm bittiğinde 9 ana terimi açıklayabilecek ve 16 ek terimi duyduğunda tanıyabileceksin.

Game Mechanic **TEMEL**

Oyuncunun oyunla etkileştiği tekil kural/eylem: zıplama, kart çekme, blok koyma gibi.

Nerede duyarsın? core mechanic.

Aklında kalsın: Mechanic tek hareket; system birden çok mechanic/verinin birlikte çalışması olabilir.

Core Loop **TEMEL**

Oyuncunun oyunda tekrar tekrar yaptığı ana eylem döngüsüdür.

Nerede duyarsın? gameplay loop, core loop.

Aklında kalsın: Örn: keşfet → savaş → ödül al → güçlen → daha zor alana git.

Game System **TEMEL**

Birden fazla kural, veri ve davranışın birlikte çalıştığı yapı: inventory, combat, economy, transfer sistemi.

Nerede duyarsın? system design.

Aklında kalsın: Kod sınıfından daha büyük tasarım birimidir.

Spawn / Despawn **TEMEL**

Bir oyun nesnesini oluşturup dünyaya eklemek / dünyadan kaldırmak için kullanılan genel terimlerdir.

Nerede duyarsın? enemy spawner.

Aklında kalsın: Godot'ta spawn çoğu zaman instantiate + add_child; despawn queue_free olabilir.

RNG / Random Seed **TEMEL**

RNG rastgele sayı üretimini; seed ise aynı rastgele diziyi yeniden üretebilmek için başlangıç değerini ifade eder.

Nerede duyarsın? random seed, procedural RNG.

Aklında kalsın: "Random" sistemlerin test edilebilirliği için seed önemli olabilir.

Weighted Random **TEMEL**

Seçeneklerin eşit değil belirli ağırlıklarla seçildiği rastgelelik yöntemidir.

Nerede duyarsın? loot rarity, weighted chance.

Aklında kalsın: Legendary %1, common %70 gibi.

Pathfinding **TEMEL**

Bir karakterin başlangıçtan hedefe geçerli yol bulması problemidir.

Nerede duyarsın? A*, navigation path.

Aklında kalsın: AI = sadece pathfinding değildir; pathfinding AI'nın araçlarından biridir.

Navigation / NavMesh **TEMEL**

Yürünebilir alanı ve üzerinde rota bulmayı temsil eden sistemdir. 3D'de navmesh terimini sık duyarsın.

Nerede duyarsın? navigation agent, navmesh bake.

Aklında kalsın: Haritanın görseli değil, AI'nın yürüyebileceği temsil.

Behavior Tree **TEMEL**

AI davranışlarını seçici, sıra, koşul ve eylem gibi düğümlerle hiyerarşik biçimde modelleyen yaygın yaklaşım.

Nerede duyarsın? BT, selector, sequence.

Aklında kalsın: Başlangıçta FSM öğrenmek daha kolay; Behavior Tree daha karmaşık AI'da çıkar.

Hızlı sözlük

Cooldown	Bir ability/eylemin tekrar kullanılmadan önce bekleme süresi.
Stat	Oyuncu/nesne sayısal özelliği: speed, attack, stamina.
Buff / Debuff	Geçici veya koşullu olumlu / olumsuz stat-etki.
Inventory	Item'ların tutulduğu ve yönetildiği sistem.
Quest	Hedefler ve ödülleri içeren görev sistemi.
Ability / Skill	Karakterin kullanabildiği özel eylem/yetenek.
Damage	Can veya başka kaynak üzerinde azalma üreten etki.
Health / HP	Can değerini temsil eden kaynak.
Stamina / Mana	Eylemleri sınırlayan tüketilebilir kaynak örnekleri.
Aggro	AI'nın hedefe düşmanlık/öncelik göstermesini anlatan oyun terimi.
Line of Sight (LOS)	Bir hedefin arada engel olmadan görülebilmesi.
Steering	AI hareketinde hedefe yönelme, kaçınma, ayrışma gibi sürekli hareket davranışları.
Procedural Generation	İçeriğin kurallar/algoritmalarla otomatik üretilmesi.
Deterministic	Aynı input/başlangıç koşullarında aynı sonucu üretme özelliği.
Hit Scan	Mermi uçuşunu simüle etmek yerine anlık ray/sorgu ile isabet belirleme yaklaşımı.
Projectile	Dünyada hareket eden mermi/ok gibi gerçek oyun nesnesi.

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlelerle söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. Core loop ile mechanic farkı nedir?
2. RNG seed neden faydalıdır?
3. Pathfinding AI'nın tamamı mıdır?

BÖLÜM 11

Audio, networking ve multiplayer sözlüğü

Şimdilik derinlemesine değil; duyunca ne olduğunu bilmen yeterli.

HEDEF Bu bölüm bittiğinde 8 ana terimi açıklayabilecek ve 13 ek terimi duyduğunda tanıyabileceksin.

Audio Bus TEMEL

Sesleri gruplandırıp volume/effect gibi işlemleri topluca uyguladığın mixer kanalıdır.

Nerede duyarsın? Master bus, SFX bus, music bus.

Aklında kalsın: Music ve SFX'i ayrı bus'larda tutmak yaygın pratiktir.

SFX / BGM TEMEL

SFX ses efektleri; BGM arka plan müziği için kullanılan yaygın kısaltmalardır.

Nerede duyarsın? sound effects, background music.

Aklında kalsın: Audio konuşmalarının temel kısaltmaları.

Client / Server TEMEL

Client oyuncunun uygulaması; server oyunun ortak durumunu barındıran/koordine eden taraf olarak kullanılan temel ağ modelidir.

Nerede duyarsın? dedicated server, client prediction.

Aklında kalsın: Multiplayer'da "kime güveniyoruz?" sorusu merkezidir.

Peer TEMEL

Ağdaki katılımcı/bağlantı uçlarından her biri için kullanılan genel terimdir.

Nerede duyarsın? multiplayer peer.

Aklında kalsın: Client-server modelinde bile peer kelimesi API'de karşına çıkabilir.

RPC TEMEL

Remote Procedure Call: bir taraftan başka ağ peer'ında fonksiyon/işlem çalıştırma mekanizmasıdır.

Nerede duyarsın? rpc call, @rpc.

Aklında kalsın: Normal fonksiyon çağrısının ağ üzerinden uzaktaki eşine yönelmiş hali gibi düşün.

Authority TEMEL

Bir ağ nesnesinin "doğru kabul edilen" durumunu hangi peer'ın kontrol ettiğini belirleyen yetki kavramıdır.

Nerede duyarsın? server authority, multiplayer authority.

Aklında kalsın: Hile önleme ve tutarlılık için önemlidir.

Latency / Ping TEMEL

Verinin ağda gidip gelme gecikmesidir. Ping genellikle round-trip gecikmeyi ms cinsinden ifade eder.

Nerede duyarsın? high ping, latency compensation.

Aklında kalsın: 60 FPS hızlı olsa bile ağ gecikmesi ayrı problemdir.

Replication / Synchronization **TEMEL**

Oyun durumunun ağdaki peer'lar arasında tutarlı biçimde çoğaltılması/senkronize edilmesidir.

Nerede duyarsın? state replication.

Aklında kalsın: Konum, animasyon, can gibi veriler hangi sıklıkla ve kimden kime gider?

Hızlı sözlük

dB (Decibel)	Ses seviyelerini ifade ederken kullanılan logaritmik birim; 0 dB "maksimum ses" anlamına gelmek zorunda değildir ama dijital mixer bağlamında referans seviyedir.
Audio Stream	Çalınabilir ses verisini temsil eden kaynak türü fikri.
Loop	Ses/müzik bittikten sonra tekrar başlatma.
Spatial Audio	Sesin 2D/3D konuma göre yön ve mesafe hissi vermesi.
Packet	Ağ üzerinden gönderilen veri birimi.
Bandwidth	Belirli sürede taşınabilecek veri miktarı.
Packet Loss	Gönderilen paketlerin bir kısmının hedefe ulaşmaması.
Jitter	Paket gecikmesindeki düzensiz dalgalanma.
Dedicated Server	Oyuncu olmayan, sadece server rolü çalışan ayrı süreç/makine.
Host	Bir multiplayer oturumunu başlatan ve bazen server rolünü de üstlenen peer.
Prediction	Client'ın server cevabı gelmeden kendi hareket sonucunu tahmin ederek gecikmeyi gizleme tekniği.
Interpolation	Uzaktan gelen seyrek state'ler arasında yumuşak görsel hareket üretme.
Rollback	Özellikle hızlı multiplayer oyunlarda geçmiş state'e dönüp yeni inputla tekrar simüle etme yaklaşımı.

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlele söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. RPC neyin kısaltmasıdır ve ne işe yarar?
2. Latency ile FPS aynı problem midir?
3. Authority neden önemlidir?

BÖLÜM 12

Debugging, performance ve üretim araçları

"Kod çalışmıyor"dan "neden çalışmadığını biliyorum" seviyesine.

HEDEF Bu bölüm bittiğinde 9 ana terimi açıklayabilecek ve 14 ek terimi duyduğunda tanıyabileceksin.

Bug **TEMEL**

Programın beklenen davranıştan sapmasına neden olan hata/problem için genel terimdir.

Nerede duyarsın? bug fix, regression bug.

Aklında kalsın: Bug = her zaman syntax hatası değildir.

Syntax / Runtime / Logic Error **TEMEL**

Syntax dil kurallarına uymayan kod; runtime çalışırken oluşan hata; logic error ise kod çalışsa da yanlış sonuç üretmesidir.

Nerede duyarsın? parser error, runtime exception.

Aklında kalsın: Hata türünü tanımak çözümü hızlandırır.

Debugger / Breakpoint **TEMEL**

Debugger çalışan programın durumunu incelemene yarar. Breakpoint kodu belirli satırda durdurup değişkenleri adım adım görmeni sağlar.

Nerede duyarsın? set breakpoint, step over.

Aklında kalsın: print() tek araç değildir.

Stack Trace **TEMEL**

Hata anına gelene kadar hangi fonksiyon çağrılarından geçildiğini gösteren izdir.

Nerede duyarsın? call stack.

Aklında kalsın: İlk hata mesajıyla birlikte stack trace'i oku.

Profiler **TEMEL**

CPU zamanı, script fonksiyon süreleri ve performans davranışını ölçmene yardım eden araçtır.

Nerede duyarsın? profile the game.

Aklında kalsın: Tahminle değil ölçümle optimize et.

Bottleneck **TEMEL**

Sistemin genel performansını en çok sınırlayan dar boğazdır.

Nerede duyarsın? CPU bottleneck, GPU bottleneck.

Aklında kalsın: En yavaş kritik bölüm iyileştirilmeden başka yerleri optimize etmek az sonuç verebilir.

CPU vs GPU **TEMEL**

CPU genel oyun mantığı, fizik, script vb.; GPU görüntü çizimi ve paralel grafik hesaplarında öne çıkar.

Nerede duyarsın? CPU-bound, GPU-bound.

Aklında kalsın: Performans problemi "oyun yavaş" demekle çözülmez; hangi taraf sınır?

Memory / Allocation **TEMEL**

Memory runtime verisinin tutulduğu bellek; allocation yeni veri/nesne için bellek ayırma işlemidir.

Nerede duyarsın? memory usage, allocations.

Aklında kalsın: Sık oluşturma-sil döngüleri bazı durumlarda maliyetli olabilir.

Premature Optimization **TEMEL**

Gerçek darboğazı ölçmeden önce kodu gereksiz karmaşıklığa sokarak optimizasyonlara girişmektir.

Nerede duyarsın? don't optimize prematurely.

Aklında kalsın: Önce doğru ve anlaşılır çalıştır; sonra profiler.

Hızlı sözlük

Log	Programın çalışma bilgilerini yazdığı kayıt çıktısı.
print()	Basit runtime değerlerini görmek için kullanılan çıktı fonksiyonu.
Warning	Kod çalışabilir ama şüpheli/istenmeyen durum bildiren uyarı.
Regression	Daha önce çalışan bir özelliğin yeni değişiklikten sonra bozulması.
Reproduce	Bir bug'ı tekrar aynı koşullarda ortaya çıkarabilmek.
Minimal Reproduction	Bug'ı gösteren mümkün olan en küçük proje/kod örneği.
FPS Monitor	Frame rate'i izleme metriği.
Frame Time	Bir frame üretmek için geçen süre; ms cinsinden bakmak performansı anlamayı kolaylaştırır.
Culling	Görünmeyecek nesneleri render işinden çıkarmak.
LOD	Level of Detail: uzaktaki nesnelerde daha ucuz detay seviyesi kullanma.
Batching	Birden çok çizim işini daha az render komutunda birleştirmeye yönelik teknikler.
Cache	Tekrar kullanılacak veriyi daha hızlı erişim için saklama.
Benchmark	Belirli senaryoda performansı ölçüp karşılaştırma testi.
Hot Path	Programın çok sık çalıştığı veya çok zaman harcadığı kritik kod yolu.

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlelerle söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. Logic error neden syntax error'dan farklıdır?
2. Profiler kullanmanın ana amacı nedir?
3. Stack trace sana ne anlatır?

BÖLÜM 13

Git, proje düzeni, build ve yayınlama

Tek başına çalışsan bile erken öğrenmen avantaj sağlar.

HEDEF Bu bölüm bittiğinde 8 ana terimi açıklayabilecek ve 13 ek terimi duyduğunda tanıyabileceksin.

Version Control TEMEL

Dosya değişikliklerinin geçmişini kaydedip karşılaştırma, geri dönme ve ekipçe çalışma sistemidir. Git en yaygın örneklerden biridir.

Nerede duyarsın? source control, VCS.

Aklında kalsın: "Dün çalışan haline dönmek" için hayat kurtarır.

Commit TEMEL

Projedeki belirli değişiklik grubunun isimlendirilmiş/sabitlenmiş kayıt noktasıdır.

Nerede duyarsın? make a commit.

Aklında kalsın: Küçük ve anlamlı commit'ler hata ayıklamayı kolaylaştırır.

Branch TEMEL

Ana geliştirmeden ayrılan paralel değişiklik çizgisidir.

Nerede duyarsın? feature branch, main branch.

Aklında kalsın: Yeni özelliği ayrı branch'te deneyip sonra birleştirebilirsin.

Merge TEMEL

İki branch'teki değişiklikleri birleştirme işlemidir.

Nerede duyarsın? merge conflict.

Aklında kalsın: Aynı satırlar değiştiyse conflict çıkabilir.

.gitignore TEMEL

Git'in takip etmemesi gereken geçici/cache/build dosyalarını tarif eden dosyadır.

Nerede duyarsın? ignore .godot folder.

Aklında kalsın: Motor tarafından yeniden üretilebilen cache dosyaları çoğu zaman repoya konmaz.

Debug vs Release Build TEMEL

Debug build geliştirme ve hata ayıklama bilgilerine odaklanır; release build dağıtım/performans için daha uygun ayarlarla hazırlanır.

Nerede duyarsın? export debug, export release.

Aklında kalsın: Oyuncuya debug build göndermek her zaman doğru değildir.

Export Preset TEMEL

Godot'ta hedef platform ve export seçeneklerini tanımlayan kayıtlı yapılandırmadır.

Nerede duyarsın? Windows Desktop preset.

Aklında kalsın: Her platform için farklı preset olabilir.

CI/CD TEMEL

Kod değişikliklerinden sonra test/build/export gibi adımları otomatik çalıştıran süreçlerin genel adı.

Nerede duyarsın? GitHub Actions, build pipeline.

Aklında kalsın: Başlangıçta şart değil; ekip/proje büyüdükçe duyarsın.

Hızlı sözlük

Repository	Git ile takip edilen proje deposu.
Remote	Repo'nun GitHub/GitLab gibi uzaktaki kopyası.
Push	Yerel commit'leri remote'a gönderme.
Pull	Remote değişikliklerini yerel repoya alma/birleştirme.
Clone	Remote repoyu yerel bilgisayara kopyalama.
Diff	İki sürüm arasındaki değişiklikleri gösteren karşılaştırma.
Conflict	Git otomatik birleştiremediğinde manuel karar gerektiren çakışma.
Tag	Belirli commit'e sürüm gibi anlamlı sabit etiket verme.
Semantic Versioning	Sürümü major.minor.patch biçiminde ifade eden yaygın yaklaşım: 1.4.2.
Release	Kullanıcıya dağıtılması amaçlanan belirli ürün sürümü.
Patch	Küçük hata düzeltmesi/güncelleme paketi veya sürümü.
Hotfix	Üretimde acil kritik sorun için hızlı düzeltme.
Backup	Version control'den bağımsız ek veri kopyası; ikisi aynı şey değildir.

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlele söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. Commit nedir?
2. Version control ile backup aynı şey midir?
3. Debug ve release build arasında genel fark nedir?

BÖLÜM 14

Oyun tasarımı ve proje üretim dili

Kod bilmek kadar “ne üretiyoruz?” dilini de tanımak gerekir.

HEDEF Bu bölüm bittiğinde 9 ana terimi açıklayabilecek ve 17 ek terimi duyduğunda tanıyabileceksin.

Prototype TEMEL

Bir fikrin çalışıp çalışmadığını hızlı ve ucuz biçimde test eden erken sürümdür.

Nerede duyarsın? prototype the mechanic.

Aklında kalsın: Güzel görünmek değil, soruya cevap vermek için yapılır.

Vertical Slice TEMEL

Oyunun küçük ama hedef kaliteyi temsil eden uçtan uca tamamlanmış parçasıdır.

Nerede duyarsın? vertical slice demo.

Aklında kalsın: Prototype “fikir çalışıyor mu?”, vertical slice “final kaliteye yakın üretim nasıl görünüyor?” sorusuna yaklaşır.

MVP TEMEL

Minimum Viable Product: temel değer önerisini çalışır biçimde sunan minimum kapsamlı ürün sürümüdür.

Nerede duyarsın? MVP scope.

Aklında kalsın: Game dev’de terim farklı ekiplerde farklı sertlikte kullanılabilir.

Scope TEMEL

Projede yapılacak işin kapsamı ve sınırlarıdır.

Nerede duyarsın? scope creep, cut scope.

Aklında kalsın: Yeni başlayan projelerinin en büyük risklerinden biri aşırı scope.

Iteration TEMEL

Bir şeyi yap, test et, geri bildirim al, düzelt ve tekrar et döngüsüdür.

Nerede duyarsın? iterate on combat.

Aklında kalsın: İlk sürümün final olması beklenmez.

Playtest TEMEL

Gerçek oyuncu davranışını gözlemek için oyunu oynatıp veri/geri bildirim toplama sürecidir.

Nerede duyarsın? playtest session.

Aklında kalsın: “Ben oynadım, güzel” playtest değildir.

Balancing TEMEL

Mekanik, ekonomi, zorluk ve değerleri istenen deneyime göre ayarlama sürecidir.

Nerede duyarsın? balance damage, economy balance.

Aklında kalsın: Her şeyin eşit olması balancing değildir.

Game Feel / Juice TEMEL

Inputa verilen görsel, sesli, animasyonlu geri bildirimlerle eylemlerin tatmin edici hissedilmesi. "Juice" daha çok ekstra geri bildirim katmanlarını anlatır.

Nerede duyarsın? screen shake, hit stop, juice.

Aklında kalsın: Aynı mechanic, iyi feedback ile çok daha güçlü hissedebilir.

Feedback / Affordance TEMEL

Feedback oyuncunun eylemine sistem cevabıdır. Affordance bir objenin ne yapılabilir olduğunu görünüş/işaretlerle sezdirme fikridir.

Nerede duyarsın? visual feedback, button affordance.

Aklında kalsın: Oyuncu "ne oldu?" veya "ne yapabilirim?" diye kalmamalı.

Hızlı sözlük

Feature	Oyundaki belirli işlev/özellik.
Milestone	Proje boyunca hedeflenen önemli teslim veya aşama.
Roadmap	Zaman içinde hangi özellik/aşamaların planlandığını gösteren genel yol haritası.
Backlog	Henüz yapılmamış iş/özellik/fix listesinin havuzu.
Task	Yapılabilir ölçekte tek iş maddesi.
Bugfix	Bir hatayı düzeltmeye yönelik değişiklik.
Scope Creep	Planlanan kapsamın kontrolsüz biçimde büyümesi.
Cut	Bir özelliği/kapsamı projeden çıkarmak.
Polish	Temel sistemler çalıştıktan sonra detay, tutarlılık ve his iyileştirmeleri.
Onboarding	Oyuncunun oyuna, kontrollere ve sistemlere alışmasını sağlayan ilk deneyim.
Tutorial	Oyuncuya mekanik/sistem öğretme bölümü veya yöntem.
Difficulty Curve	Zorluğun oyun boyunca nasıl arttığını/azaldığını ifade eden yapı.
Progression	Oyuncunun zamanla güç, içerik, yetenek veya statü kazanma yapısı.
Fail State / Win State	Kaybetme ve kazanma koşulları.
UX	User Experience: kullanıcının sistemi kullanırken yaşadığı bütün deneyim.
UI	UX'in görsel/etkileşimsel arayüz katmanı; UX ile aynı şey değildir.
Accessibility	Farklı fiziksel/duyusal/bilişsel ihtiyaçlara sahip oyuncuların deneyime erişebilmesini destekleyen tasarım.

Mini kontrol

Kural: Cevabı ezberden değil, kendi cümlelerle söylemeye çalış. 60 saniyede açıklayamıyorsan terimi bir kez daha oku.

1. Prototipe ile vertical slice arasındaki fark nedir?
2. Scope creep ne demektir?
3. UI ve UX aynı şey midir?

BÖLÜM 15

Alfabetik büyük sözlük

Hızlı bakış için: bu kitapta geçen ana ve yardımcı terimler tek yerde.

.gitignore — Git'in takip etmemesi gereken geçici/cache/build dosyalarını tarif eden dosyadır.

@export — Bir property'yi Inspector'da düzenlenebilir hale getiren annotation.

@onready — Node hazır olduğunda bir değişkeni initialize etmeyi kolaylaştıran annotation.

_physics_process(delta) — Sabit fizik adımlarına yakın aralıklarla çalışan callback'tir. CharacterBody hareketi ve fizik ilişkili hesaplar burada yapılır.

_process(delta) — Her çizim frame'inde çalışabilen callback'tir. UI, görsel takip, frame-temelli işler için sık kullanılır.

_ready() — Node SceneTree'ye girip çocukları hazır olduğunda çağrılan temel lifecycle callback'lerinden biridir.

AABB — Axis-Aligned Bounding Box; eksenlere hizalı sınırlayıcı kutu, özellikle 3D geometri/çarpışma optimizasyonunda çıkar.

Ability / Skill — Karakterin kullanabildiği özel eylem/yetenek.

Abstraction — Gereksiz iç ayrıntıları saklayıp önemli davranışı daha basit arayüzle sunma.

Accessibility — Farklı fiziksel/duyusal/bilişsel ihtiyaçlara sahip oyuncuların deneyime erişebilmesini destekleyen tasarım.

Add-on — Godot'a editor veya runtime özelliği ekleyen eklenti paketi.

add_child() — Runtime'da bir Node'u SceneTree'ye child olarak ekler.

Aggro — AI'nın hedefe düşmanlık/öncelik göstermesini anlatan oyun terimi.

Alpha — Renk/texture saydamlık kanalı.

Anchor / Offset — Anchor parent boyutuna göre oransal referans; offset ise bu referansa eklenen piksel uzaklığıdır.

Angle — İki yön arasındaki açısal değer.

AnimationPlayer — Property değerlerini zaman içinde keyframe'lerle değiştiren genel animasyon aracıdır.

AnimationTree — Daha karmaşık animasyon blend ve state geçişlerini yöneten Godot sistemi.

Annotation — GDScript'te @ ile başlayan özel işaretler: @export, @onready.

Anti-aliasing — Tırtıklı kenarları yumuşatma tekniklerinin genel adı.

API — Bir sistemin koddan kullanıma izin verdiği sınıf, metod, property ve kurallar bütünüdür.

Area — Fiziksel olarak duvar oluşturmak yerine başka body/area giriş-çıkışlarını algılayan bölgedir.

Array / Dictionary — Array sıralı değer listesi; Dictionary anahtar-değer eşleşmesi tutar.

Aspect Ratio — Ekranın genişlik/yükseklik oranıdır: 16:9, 21:9 gibi.

Asset — Oyunda kullanılan içerik dosyalarının genel adıdır: görsel, ses, font, 3D model, animasyon, veri dosyası vb.

Asset Pipeline — Assetlerin üretilmesi, import edilmesi, işlenmesi ve projede kullanılmasına kadar olan süreç.

Atlas / Sprite Sheet — Birden fazla küçük görseli tek büyük texture içinde saklama yaklaşımıdır.

Audio Bus — Sesleri gruplandırıp volume/effect gibi işlemleri topluca uyguladığın mixer kanalıdır.

Audio Stream — Çalınabilir ses verisini temsil eden kaynak türü fikri.

Authority — Bir ağ nesnesinin "doğru kabul edilen" durumunu hangi peer'in kontrol ettiğini belirleyen yetki kavramıdır.

Autoload — Bir scene veya scripti proje boyunca otomatik yüklenen global erişilebilir node olarak kullanma mekanizmasıdır.

await — Bir signal veya coroutine benzeri işlemin tamamlanmasını bekleyip sonra devam etme.

Axis — x, y, z yönlerinden biri.

Backlog — Henüz yapılmamış iş/özellik/fix listesinin havuzu.

Backup — Version control'den bağımsız ek veri kopyası; ikisi aynı şey değildir.

Bake — Önceden hesaplanabilen bir veriyi (navigation, light vb.) çalışma öncesinde üretmek.

Balancing — Mekanik, ekonomi, zorluk ve değerleri istenen deneyime göre ayarlama sürecidir.

Bandwidth — Belirli sürede taşınabilecek veri miktarı.

Batching — Birden çok çizim işini daha az render komutunda birleştirmeye yönelik teknikler.

Behavior Tree — AI davranışlarını seçici, sıra, koşul ve eylem gibi düğümlerle hiyerarşik biçimde modelleyen yaygın yaklaşım.

Benchmark — Belirli senaryoda performansı ölçüp karşılaştırma testi.

Blend — İki animasyon veya değeri oranlı şekilde karıştırma.

Blend Tree — Animasyonları parametrelere göre karıştıran yapı; farklı motorlarda benzer terimler kullanılır.

bool — true / false.

Bottleneck — Sistemin genel performansını en çok sınırlayan dar boğazdır.

Bounce / Restitution — Çarpışmada ne kadar sekme olacağını belirleyen fizik özelliği.

Bounding Box — Bir nesnenin kapladığı alanı yaklaşık çevreleyen kutu.

Branch — Ana geliştirmeden ayrılan paralel değişiklik çizgisidir.

Buff / Debuff — Geçici veya koşullu olumlu / olumsuz stat-etki.

Bug — Programın beklenen davranıştan sapmasına neden olan hata/problem için genel terimdir.

Bugfix — Bir hatayı düzeltmeye yönelik değişiklik.

Build / Export — Projenin oyuncunun çalıştırabileceği platform çıktısına dönüştürülmesidir. Godot genellikle "Export" kelimesini kullanır.

Cache — Tekrar kullanılacak veriyi daha hızlı erişim için saklama.

Callable — Sonradan çağrılacak bir fonksiyonu temsil eden Godot türü.

Camera — Oyuncuya dünyanın hangi bölümünün ve nasıl gösterileceğini belirler.

Canvas — Godot 2D çizim dünyasının temel render katmanı kavramı.

CanvasLayer — Godot'ta 2D çizimleri farklı canvas katmanında tutmaya yarayan node; HUD için sık kullanılır.

Casting — Bir değeri başka bir tipe dönüştürme veya o tipe ele alma.

CharacterBody — Kodla kontrol edilen oyuncu/NPC karakterleri gibi hareketli gövdeler için tasarlanmış body türüdür.

CI/CD — Kod değişikliklerinden sonra test/build/export gibi adımları otomatik çalıştıran süreçlerin genel adı.

Clamp — Bir değeri minimum ve maksimum sınır arasında tutma.

Class / Object / Instance — Class bir tür/şablon fikridir; object o türden çalışan nesnedir; instance ise bir sınıfın veya sahnenin oluşturulmuş örneğidir.

class_name — Script sınıfına global olarak kullanılabilen isim verir.

Client / Server — Client oyuncunun uygulaması; server oyunun ortak durumunu barındıran/koordine eden taraf olarak kullanılan temel ağ modelidir.

Clone — Remote repoyu yerel bilgisayara kopyalama.

Cohesion — Bir sınıf veya modüldeki parçaların tek ve anlamlı bir sorumluluk etrafında ne kadar iyi toplandığını anlatır.

Collision — İki fizik şeklinin birbirine temas/örtüşme durumunun motor tarafından algılanmasıdır.

Collision Layer / Mask — Layer "ben hangi fizik katmanındayım?", mask ise "hangi katmanları algılayacağım?" mantığıdır.

CollisionShape2D / 3D — Bir fizik nesnesinin çarpışma geometrisini tanımlar. Rectangle, capsule, sphere vb. şekiller kullanılabilir.

Commit — Projedeki belirli değişiklik grubunun isimlendirilmiş/sabitlenmiş kayıt noktasıdır.

Component — Tek sorumluluklu, başka nesneye eklenerek davranış kazandıran modüler parça fikri.

Composition — Davranışları tek dev sınıf yerine küçük parçaları birleştirerek kurma yaklaşımı.

Composition over Inheritance — Davranışları derin inheritance zincirleri yerine küçük parçaları birleştirerek kurmayı tercih eden tasarım ilkesi.

Conditional — Koşula göre farklı kod yollarını çalıştırır. if / elif / else ve match bu amaçla kullanılır.

Config — Ayar verilerini temsil eden genel kavram.

Conflict — Git otomatik birleştiremediğinde manuel karar gerektiren çakışma.

Constant (Sabit) — Program çalışırken değiştirilmemesi amaçlanan değerdir. GDScript'te const ile tanımlanır.

Container — Child Control node'larının boyut/konumunu otomatik düzenleyen UI node'larıdır. VBox, HBox, Grid, Margin gibi çeşitleri vardır.

Control — Godot'ta UI node'larının temel sınıf ailesidir. Position/size yerine anchors, offsets ve containers gibi UI düzen mantıklarıyla çalışır.

Cooldown — Bir ability/eylemin tekrar kullanılmadan önce bekleme süresi.

Coordinate System — Dünyadaki konumu eksenlerle ifade eden sistemdir. 2D'de x/y, 3D'de x/y/z kullanılır.

Core Loop — Oyuncunun oyunda tekrar tekrar yaptığı ana eylem döngüsüdür.

Coroutine — Çalışması zaman içinde duraklayıp daha sonra devam edebilen fonksiyon akışı için genel terim.

Coupling — İki sistemin birbirine ne kadar sıkı bağımlı olduğunu anlatır.

CPU vs GPU — CPU genel oyun mantığı, fizik, script vb.; GPU görüntü çizimi ve paralel grafik hesaplarında öne çıkar.

Cross Product — 3D'de vectorlere dik yön üretme gibi işlemlerde kullanılan çarpım.

Culling — Görünmeyecek nesneleri render işinden çıkarmak.

Cut — Bir özelliği/kapsamı projeden çıkarmak.

Damage — Can veya başka kaynak üzerinde azalma üreten etki.

Data Type (Veri Tipi) — Bir değerin ne tür veri olduğunu belirtir: int, float, bool, String, Vector2 gibi.

Data vs Logic — Data "ne var?" bilgisidir; logic "bu bilgiyle ne yapılır?" davranışdır. Oyuncu speed=80 data; calculate_speed() logic örneğidir.

dB (Decibel) — Ses seviyelerini ifade ederken kullanılan logaritmik birim; 0 dB "maksimum ses" anlamına gelmek zorunda değildir ama dijital mixer bağlamında referans seviyedir.

Dead Zone — Analog stick küçük titreşimlerinin input sayılmaması için merkez çevresindeki tolerans.

Debug vs Release Build — Debug build geliştirme ve hata ayıklama bilgilerine odaklanır; release build dağıtım/performans için daha uygun ayarlarla hazırlanır.

Debugger / Breakpoint — Debugger çalışan programın durumunu incelemene yarar. Breakpoint kodu belirli satırda durdurup değişkenleri adım adım görmeni sağlar.

Dedicated Server — Oyuncu olmayan, sadece server rolü çalışan ayrı süreç/makine.

Degree / Radian — Açıyı ifade eden iki birim. Programlama API'lerinde radyan sık kullanılır.

delta — Son güncelleme adımından beri geçen süreyi saniye cinsinden temsil eden değerdir.

Dependency — Bir sistemin çalışmak için ihtiyaç duyduğu başka paket, sınıf veya servis.

Dependency Injection — Bir nesnenin ihtiyacını kendisi yaratmak yerine dışarıdan alması yaklaşımı.

Deterministic — Aynı input/başlangıç koşullarında aynı sonucu üretme özelliği.

Diff — İki sürüm arasındaki değişiklikleri gösteren karşılaştırma.

Difficulty Curve — Zorluğun oyun boyunca nasıl arttığını/azaldığını ifade eden yapı.

Distance / Direction — İki nokta arasındaki mesafe ve birinden diğerine yön oyun mantığında çok sık hesaplanır.

Dot Product — İki vector'ün yön benzerliğini ölçmede kullanılan işlem; görüş konisi vb. ileri konularda çıkar.

DPI — Fiziksel ekran yoğunluğuyla ilişkili ölçü; özellikle mobil ve yüksek yoğunluklu ekranlarda UI ölçekleme konuşmalarında çıkar.

Draw Call — CPU'nun GPU'ya "şu geometriyi/öğeyi çiz" şeklinde gönderdiği render komutlarının genel terimidir.

Draw Order — 2D/3D öğelerin hangi sırada çizildiği.

Dynamic vs Static Typing — Dynamic kullanımda tip çalışma anında belirlenebilir. Static typing'de değişken/parametre/return tipi açıkça belirtilir. GDScript ikisini de destekler.

Easing — Animasyon hızının başta/ortada/sonda nasıl değişeceğini tanımlayan eğri/karakter.

Editor — Motorun görsel çalışma alanıdır. Sahne düzenler, node ekler, Inspector'dan değer değiştirir, script açar ve oyunu çalıştırır.

Editor Tool Script — Godot editor içinde çalışarak özel araç üretmeye yarayan script yaklaşımı.

Encapsulation — Bir sistemin iç detaylarını saklayıp dışarıya kontrollü bir arayüz sunma fikridir.

Enum — İsimlendirilmiş sabit seçenek kümesi: IDLE, RUNNING, DEAD.

Event — Bir şeyin gerçekleştiğini temsil eden bildirim/veri; Godot'ta signal ile sık ilişkilidir.

Event Bus — Birçok sistemin ortak olay kanalı üzerinden haberleşmesi yaklaşımı.

Export Preset — Godot'ta hedef platform ve export seçeneklerini tanımlayan kayıtlı yapılandırma.

Expression — Bir değere hesaplanan kod parçası: damage * 2 + bonus.

extends — Bir scriptin hangi Godot sınıfından veya custom class'tan türediğini belirtir.

Factory — Nesne oluşturma kararlarını tek yerde toplayan tasarım yaklaşımı.

Fail State / Win State — Kaybetme ve kazanma koşulları.

Feature — Oyundaki belirli işlev/özellik.

Feedback / Affordance — Feedback oyuncunun eylemine sistem cevabıdır. Affordance bir objenin ne yapılabilir olduğunu görünüş/işaretlerle sezdirme fikridir.

Filtering — Texture büyütülür/küçültülürken piksellerin nasıl örnekleneceği.

Finite State Machine (FSM) — Bir sistemi sınırlı state'ler ve aralarındaki geçişlerle modelleyen yaklaşımdır.

Fixed Timestep — Özellikle fizik için sabit aralıklarla güncelleme yaklaşımı.

float — Ondalıklı sayı: 3.5, 0.016.

Focus — UI'da hangi Control'un klavye/gamepad inputunu alacağını belirleyen durum.

Focus Navigation — Klavye/gamepad ile UI elemanları arasında odak geçişi.

Force — Bir rigid body'nin hareketini zaman içinde değiştiren kuvvet.

FOV — 3D kameranın görüş açısı (field of view).

FPS Monitor — Frame rate'i izleme metriği.

Frame / FPS — Frame ekrana üretilen bir görüntü karesidir. FPS saniyedeki frame sayısıdır.

Frame Independent — Davranışın FPS değişse bile yaklaşık aynı gerçek zaman hızında ilerlemesi.

Frame Time — Bir frame üretmek için geçen süre; ms cinsinden bakmak performansı anlamayı kolaylaştırır.

Friction — Temas eden yüzeylerin kaymaya karşı direnci.

Function / Method — Belirli işi yapan tekrar kullanılabilir kod bloğudur. Bir nesne/sınıf bağlamındaki fonksiyona genellikle method denir.

Game Engine (Oyun Motoru) — Grafik, input, fizik, ses, sahne yönetimi ve platforma çıktı alma gibi temel işleri hazır sunan geliştirme ortamıdır. Godot, Unity ve Unreal birer oyun motorudur.

Game Feel / Juice — Inputa verilen görsel, sesli, animasyonlu geri bildirimlerle eylemlerin tatmin edici hissedilmesi. "Juice" daha çok ekstra geri bildirim katmanlarını anlatır.

- Game Loop** — Oyun çalışırken input alma, dünya durumunu güncelleme, fizik ve çizim gibi adımların sürekli tekrarlanmasıdır.
- Game Mechanic** — Oyuncunun oyunla etkileştiği tekil kural/eylem: zıplama, kart çekme, blok koyma gibi.
- Game System** — Birden fazla kural, veri ve davranışın birlikte çalıştığı yapı: inventory, combat, economy, transfer sistemi.
- GDExtension** — Godot'a native kodla (ör. C++) yüksek performanslı sınıf/özellik ekleme sistemi.
- Global State** — Oyunun birden çok sistemi/sahnesi tarafından paylaşılan genel durum verisi.
- Gravity** — Nesneleri belirli yönde hızlandıran yerçekimi etkisi.
- Group** — Node'ları hiyerarşiden bağımsız etiketleyip topluca bulmak/işlemek için kullanılan sistemdir.
- Health / HP** — Can değerini temsil eden kaynak.
- Hit Scan** — Mermi uçuşunu simüle etmek yerine anlık ray/sorgu ile isabet belirleme yaklaşımı.
- Hitbox** — Genellikle saldırının vurduğu alan.
- Host** — Bir multiplayer oturumunu başlatan ve bazen server rolünü de üstlenen peer.
- Hot Path** — Programın çok sık çalıştığı veya çok zaman harcadığı kritik kod yolu.
- Hot Reload** — Bazı kod/asset değişikliklerinin uygulamayı tamamen kapatmadan yenilenebilmesi yaklaşımı.
- Hotfix** — Üretimde acil kritik sorun için hızlı düzeltme.
- HUD** — Oyun sırasında sürekli veya sık görülen bilgi arayüzü: can, skor, minimap, ammo vb.
- Hurtbox** — Genellikle hasar alabilen hedef alanı.
- IDE / Code Editor** — Kod yazdığın araç. Godot script editor, VS Code, Rider gibi.
- Import** — Dışarıdan gelen asseti motorun kullanacağı biçime hazırlama süreci.
- Impulse** — Kısa süreli ani momentum değişimi; "tekme" gibi.
- Inheritance** — Bir sınıfın üst sınıftan davranış/özellik alması.
- Input Action / Input Map** — Fiziksel tuşları doğrudan kodlamak yerine "move_left", "jump" gibi eylemler tanımlarsın; bunları Project Settings > Input Map'te cihaz girdilerine bağlarsın.
- InputEvent** — Klavye, mouse, gamepad vb. bir giriş olayını taşıyan Godot veri yapısı.
- Inspector** — Godot Editor'da seçili node/resource property'lerini görüntüleyip düzenlediğin panel.
- Instance** — Bir class veya scene şablonundan üretilmiş çalışan nesne.
- int** — Tam sayı: 5, -12, 100.
- Interface** — Bir nesnenin hangi davranışları sağlayacağını tanımlayan sözleşme fikri. GDScript'te klasik interface anahtar kelimesi yoktur; desen olarak uygulanabilir.
- Internationalization (i18n)** — Yazılımı farklı dil/bölgelere uygun tasarlama süreci.
- Interpolation** — Uzaktan gelen seyrek state'ler arasında yumuşak görsel hareket üretme.
- Interpolation / Lerp** — İki değer arasında yumuşak geçiş üretme yaklaşımıdır. Lerp lineer interpolasyonun yaygın biçimidir.
- Interpolation Physics** — Fizik tick'leri arasındaki görsel konumu yumuşatmaya yönelik interpolasyon yaklaşımı.
- Inventory** — Item'ların tutulduğu ve yönetildiği sistem.
- Iteration** — Bir şeyi yap, test et, geri bildirim al, düzelt ve tekrar et döngüsüdür.
- Jitter** — Paket gecikmesindeki düzensiz dalgalanma.
- JSON** — İnsan tarafından okunabilir yaygın veri değişim metin formatı.
- Keyframe** — Animasyonda belirli zamanda belirli değeri işaretleyen ana kare.
- Kinematic** — Genel oyun geliştirme dilinde kodla belirlenen hareket yaklaşımı; Godot 4'te karakter türü CharacterBody adını kullanır.
- Lambda** — İsimsiz/yerinde tanımlanan küçük fonksiyon.
- Latency / Ping** — Verinin ağda gidip gelme gecikmesidir. Ping genellikle round-trip gecikmeyi ms cinsinden ifade eder.
- Layer** — Görsel veya mantıksal öğeleri ayırmak için kullanılan katman kavramı; collision layer ile karıştırma.
- Layout** — UI öğelerinin yerleşim düzeni.
- Library / Framework / Plugin** — Library belirli işlevler sunan kod paketi; framework uygulamayı belli bir yapıda kurdurur; plugin/add-on ise var olan araca özellik ekler.
- License** — Bir yazılımın veya assetin hangi koşullarla kullanılacağını belirleyen hukuki izin.
- Lifecycle** — Bir nesnenin oluşturulma, hazır olma, güncellenme ve yok edilme aşamaları.
- Light** — Sahne aydınlatma kaynağı.
- Line of Sight (LOS)** — Bir hedefin arada engel olmadan görülebilmesi.
- Local vs Global Coordinates** — Local koordinat parent'a göre; global koordinat scene/world referansına göre konumu ifade eder.
- Localization (L10n)** — Oyunun metin/içeriğini farklı dil ve bölgelere uyarlama.
- LOD** — Level of Detail: uzaktaki nesnelerde daha ucuz detay seviyesi kullanma.
- Log** — Programın çalışma bilgilerini yazdığı kayıt çıktısı.
- Loop** — Ses/müzik bittikten sonra tekrar başlatma.

- Loop (Döngü)** — Aynı işlemi bir koleksiyon veya koşul üzerinden tekrarlar. for ve while temel döngülerdir.
- Magnitude / Length** — Bir vector'ün büyüklüğüdür. Örneğin hız vector'ünün length'i gerçek hız miktarını verebilir.
- Main Scene** — Proje çalıştırıldığında başlangıç olarak açılan scene.
- Manager** — Belirli bir sistemin koordinasyonundan sorumlu sınıf/node için yaygın isim; gereğinden fazla manager üretmemeye dikkat.
- Margin / Padding** — Öğeler arasındaki veya içerik ile kenar arasındaki boşluk kavramları.
- Material / Shader** — Material bir yüzeyin nasıl çizileceğine ilişkin ayar/kaynak; shader GPU üzerinde görüntüyü nasıl hesaplayacağını belirleyen programdır.
- Memory / Allocation** — Memory runtime verisinin tutulduğu bellek; allocation yeni veri/nesne için bellek ayırma işlemidir.
- Merge** — İki branch'teki değişiklikleri birleştirme işlemidir.
- Milestone** — Proje boyunca hedeflenen önemli teslim veya aşama.
- Minimal Reproduction** — Bug'ı gösteren mümkün olan en küçük proje/kod örneği.
- Minimum Size** — Bir Control'un küçülemeyeceği tercih edilen alt boyut.
- Mipmaps** — Texture'ın uzak/küçük görünüşleri için önceden oluşturulan daha düşük çözünürlüklü seviyeler.
- Modal** — Açıkken arka plan etkileşimini kısıtlayan öncelikli pencere/dialog.
- Module** — Belirli bir özelliği/sorumluluğu kapsayan bağımsız kod bölümü.
- MVP** — Minimum Viable Product: temel değer önerisini çalışır biçimde sunan minimum kapsamlı ürün sürümüdür.
- Navigation / NavMesh** — Yürünebilir alanı ve üzerinde rota bulmayı temsil eden sistemdir. 3D'de navmesh terimini sık duyarsın.
- Node** — Godot'un temel yapı taşıdır. Sprite göstermek, ses çalmak, fizik gövdesi olmak veya UI elemanı olmak gibi belirli sorumluluklar taşır.
- Node2D** — 2D transform özellikleri olan Godot node sınıfı.
- Node3D** — 3D transform özellikleri olan Godot node sınıfı.
- NodePath / get_node** — Bir node'a ağaç içindeki yolu üzerinden ulaşma yöntemidir. \$HealthBar kısa sözdizimlerinden biridir.
- Normal** — Bir yüzeye dik yön; çarpışma yüzeyinin yönünü anlamada kullanılır.
- Normalize** — Vector'ün yönünü koruyup uzunluğunu 1 yapmaktır.
- Notification** — Godot Object/Node yaşam döngüsündeki çeşitli olayların düşük seviye bildirim sistemi.
- null** — Geçerli bir nesne/değer referansı olmadığını ifade eder.
- Object** — Godot sınıf hiyerarşisinin temel sınıflarından biri ve programlamada çalışan nesne için genel terim.
- Object Pooling** — Sık oluşturulup silinen nesneleri yeniden kullanmak için havuzda tutma tekniği.
- Onboarding** — Oyuncunun oyuna, kontrollere ve sistemlere alışmasını sağlayan ilk deneyim.
- Open Source** — Kaynak kodu erişilebilir ve lisansı izin verdiği ölçüde incelenebilir/değiştirilebilir yazılım.
- Operator** — +, -, *, /, ==, !=, >, && gibi işlem işaretleri.
- Origin** — Koordinat sisteminin başlangıç noktası.
- Owner** — Bir node'un hangi scene kaynağına ait olarak kaydedileceğini etkileyen property.
- PackedScene / instantiate()** — Kaydedilmiş bir scene kaynağını temsil eder. instantiate() ile runtime'da yeni bir scene örneği üretirsin.
- Packet** — Ağ üzerinden gönderilen veri birimi.
- Packet Loss** — Gönderilen paketlerin bir kısmının hedefe ulaşmaması.
- Parameter vs Argument** — Parameter fonksiyon tanımındaki isimdir; argument fonksiyonu çağırırken verdiği gerçek değerdir.
- Parent / Child** — SceneTree hiyerarşisindeki üst-alt node ilişkisidir. Child genellikle parent transformundan/hayat döngüsünden etkilenir.
- Particle** — Çok sayıda küçük öğeyle duman, ateş, kıvılcım vb. efekt üretme sistemi.
- Patch** — Küçük hata düzeltmesi/güncelleme paketi veya sürümü.
- Pathfinding** — Bir karakterin başlangıçtan hedefe geçerli yol bulması problemidir.
- Pause** — SceneTree veya node processing davranışının oyun duraklatıldığında değişmesi.
- Peer** — Ağdaki katılımcı/bağlantı uçlarından her biri için kullanılan genel terimdir.
- Physics Material** — Friction ve bounce gibi fizik yüzey davranışlarını tanımlayan kaynak.
- Physics Tick** — Bir fizik güncelleme adımı.
- Pipeline** — Bir içeriğin üretimden oyuna girene kadar geçtiği adımlar zinciri.
- Pixel Art** — Piksel ölçüsünün bilinçli tasarım unsuru olduğu düşük çözünürlüklü görsel stil.
- Platform** — Windows, Linux, Android, iOS, Web, konsol gibi hedef çalışma ortamı.
- Playtest** — Gerçek oyuncu davranışını gözlemek için oyunu oynatıp veri/geri bildirim toplama sürecidir.
- Polish** — Temel sistemler çalıştıktan sonra detay, tutarlılık ve his iyileştirmeleri.
- Polling vs Event-driven Input** — Polling o anki input durumunu sürekli sorar; event-driven yaklaşım gelen InputEvent'i olay olarak işler.

Position — Nesnenin konumu.

Prediction — Client'in server cevabı gelmeden kendi hareket sonucunu tahmin ederek gecikmeyi gizleme tekniği.

Premature Optimization — Gerçek darboğazı ölçmeden önce kodu gereksiz karmaşıklştıracak optimizasyonlara girişmektir.

Pressed / Just Pressed — Basılı tutuluyor mu / bu frame'de yeni mi basıldı ayrımı.

print() — Basit runtime değerlerini görmek için kullanılan çıktı fonksiyonu.

Procedural Generation — İçeriğin kurallar/algoritmalarla otomatik üretilmesi.

Profiler — CPU zamanı, script fonksiyon süreleri ve performans davranışını ölçmene yardım eden araçtır.

Progression — Oyuncunun zamanla güç, içerik, yetenek veya statü kazanma yapısı.

Project — Bir oyuna ait sahneler, scriptler, assetler ve ayarların tamamıdır. Godot'ta proje kökünde project.godot bulunur.

Projectile — Dünyada hareket eden mermi/ok gibi gerçek oyun nesnesi.

Prototype — Bir fikrin çalışıp çalışmadığını hızlı ve ucuz biçimde test eden erken sürümdür.

Pull — Remote değişikliklerini yerel repoya alma/birleştirme.

Push — Yerel commit'leri remote'a gönderme.

Quest — Hedefler ve ödülleri içeren görev sistemi.

queue_free() — Bir Node'u güvenli biçimde silinmek üzere kuyruğa alır.

RayCast / ShapeCast — Belirli yön/şekil boyunca çarpışma sorgusu yapar. Görüş, zemin kontrolü, silah isabeti gibi işlerde kullanılır.

Regression — Daha önce çalışan bir özelliğin yeni değişiklikten sonra bozulması.

Release — Kullanıcıya dağıtılması amaçlanan belirli ürün sürümü.

Remap — Bir değer aralığını başka aralığa dönüştürme.

Remote — Repo'nun GitHub/GitLab gibi uzaktaki kopyası.

remove_child() — Node'u parent'tan ayırır; tek başına silmez.

Renderer — Sahneyi ekrandaki piksellere dönüştüren grafik sistemi.

reparent() — Node'un parent'ını değiştirme işlemi.

Replication / Synchronization — Oyun durumunun ağdaki peer'lar arasında tutarlı biçimde çoğaltılması/senkronize edilmesidir.

Repository — Git ile takip edilen proje deposu.

Repository (Repo) — Proje dosyalarının version control altında tutulduğu depo.

Reproduce — Bir bug'ı tekrar aynı koşullarda ortaya çıkarabilmek.

Resolution — Ekranın piksel boyutudur: 1920x1080 gibi.

Resource — Godot'un veri ve asset taşıyan temel nesne türlerinden biridir. Custom Resource ile item, karakter, skill, kart vb. veri şablonları oluşturabilirsiniz.

Responsive UI — Farklı ekran ve pencere boyutlarına uyum sağlayan arayüz.

Return Value — Bir fonksiyonun çağırana geri verdiği sonuçtur.

RigidBody — Hareketi fizik motorunun kuvvet, impulse, gravity gibi etkilerle simüle ettiği body türüdür.

RNG / Random Seed — RNG rastgele sayı üretimini; seed ise aynı rastgele diziye yeniden üretebilmek için başlangıç değerini ifade eder.

Roadmap — Zaman içinde hangi özellik/aşamaların planlandığını gösteren genel yol haritası.

Rollback — Özellikle hızlı multiplayer oyunlarda geçmiş state'e dönüp yeni inputla tekrar simüle etme yaklaşımı.

Root Node — Bir scene'in en üst node'u.

Rotation — Nesnenin yönelimi/dönüş açısı.

RPC — Remote Procedure Call: bir taraftan başka ağ peer'ında fonksiyon/işlem çalıştırma mekanizmasıdır.

Runtime — Oyunun gerçekten çalıştığı zaman dilimi ve çalışma ortamıdır.

Safe Area — Çentik, rounded corner vb. nedeniyle UI'nın güvenli yerleşebileceği ekran alanı.

Save / Load — Oyunun kalıcı durumunu saklama ve daha sonra geri yükleme sistemidir.

Scale — Nesnenin boyut ölçeği.

Scaling / Stretch — Oyunun veya UI'nın farklı pencere boyutlarına nasıl ölçekleneceğini belirleyen yaklaşım/ayarlardır.

Scene — Bir ağaç halinde organize edilmiş node grubudur ve tekrar kullanılabilir. Kaydedildiğinde .tscn gibi bir scene dosyasına dönüşebilir.

Scene Inheritance — Bir scene'den türeyerek temel yapıyı paylaşan yeni scene oluşturma yaklaşımı.

Scene Instancing — Bir scene'i başka scene içinde örnek olarak kullanma.

SceneTree — Oyun çalışırken aktif node ağacını yöneten ana yapıdır.

Schema — Bir veri yapısında hangi alanların/tiplerin bulunacağını tanımlayan yapı.

Scope — Projede yapılacak işin kapsamı ve sınırlarıdır.

Scope (Kapsam) — Bir değişkenin kodun hangi bölümünden erişilebilir olduğunu belirler: local, sınıf/script kapsamı vb.

Scope Creep — Planlanan kapsamın kontrolsüz biçimde büyümesi.

Screen Space — Ekran/viewport koordinat uzayı.

SDK — Belirli bir platform veya servis için geliştirici araçları ve API paketleri.

Semantic Versioning — Sürümü major.minor.patch biçiminde ifade eden yaygın yaklaşım: 1.4.2.

Serialization — Runtime verisini dosyaya yazılabilecek/taşınabilecek biçime dönüştürme ve geri okuma sürecidir.

SFX / BGM — SFX ses efektleri; BGM arka plan müziği için kullanılan yaygın kısaltmalardır.

Signal — Bir Object'in "bir şey oldu" diye dinleyicilere mesaj yayınlama mekanizması.

Signal / Event-driven Architecture — Bir sistemin diğerinin içini doğrudan bilmek yerine olay yayınlayıp dinleyicilerin tepki vermesi yaklaşımıdır.

Signal Connection — Bir signal yayıldığında hangi Callable'ın çalışacağını bağlama işlemi.

Singleton — Uygulama bağlamında tek erişim noktası/tek örnek olarak tasarlanan nesne paterni; Godot Autoload ile sık ilişkilendirilir ama aynı kavram değildir.

Smoothing — Ani değişimleri daha yumuşak hale getirme genel yaklaşımı.

Spatial Audio — Sesin 2D/3D konuma göre yön ve mesafe hissi vermesi.

Spawn / Despawn — Bir oyun nesnesini oluşturup dünyaya eklemek / dünyadan kaldırmak için kullanılan genel terimlerdir.

Sprite / Texture — Texture görüntü verisidir; Sprite2D bu texture'ı 2D dünyada gösteren node türlerinden biridir.

Stack Trace — Hata anına gelene kadar hangi fonksiyon çağrılarından geçildiğini gösteren izdir.

Stamina / Mana — Eylemleri sınırlayan tüketilebilir kaynak örnekleri.

Stat — Oyuncu/nesne sayısal özelliği: speed, attack, stamina.

State — Bir nesnenin belirli andaki durumunu ifade eder: IDLE, RUNNING, ATTACKING, DEAD gibi.

Statement — Programın yürüttüğü tam talimat satırı/yapısı.

StaticBody — Hareket etmeyen çevre çarpışmaları için kullanılan body türüdür.

Steering — AI hareketinde hedefe yönelme, kaçınma, ayrışma gibi sürekli hareket davranışları.

String — Metin verisi.

StyleBox — Godot Theme sisteminde panel/buton arka planı, border ve radius gibi stil kaynağı.

Syntax / Runtime / Logic Error — Syntax dil kurallarına uymayan kod; runtime çalışırken oluşan hata; logic error ise kod çalışsa da yanlış sonuç üretmesidir.

Tag — Belirli commit'e sürüm gibi anlamlı sabit etiket verme.

Task — Yapılabilir ölçekte tek iş maddesi.

Texture Atlas — Birçok küçük texture'ı tek büyük texture'da birleştirme.

Theme — UI öğelerinin font, renk, stylebox, ikon vb. görünüm kurallarını merkezi biçimde tanımlayan sistemdir.

Thread — Kod işini işletim sistemi seviyesinde paralel yürütmeye yarayan execution birimi; ileri seviye concurrency konusu.

Thread-safe — Bir kodun birden fazla thread'den eşzamanlı çağrıldığında güvenli davranabilmesi.

Tile / TileMap — Tile tekrar kullanılabilir parça; TileMap bu parçalarla grid tabanlı 2D dünya oluşturmaya yarayan sistemdir.

Time Scale — Oyunun zaman akışını hızlandırmak/yavaşlatmak için kullanılan ölçek fikri.

Timer / Cooldown — Timer belirli süre sonra veya aralıklı olay üretir. Cooldown ise bir aksiyonun tekrar kullanılmadan önce bekleme süresidir.

Timestep — Simülasyonun her güncellemede ilerlettiği zaman miktarı.

Tool / Tooling — Üretimi kolaylaştıran editör, profiler, importer, özel araç ve otomasyonların genel adı.

Tooltip — Hover/focus ile gösterilen yardımcı açıklama.

Transform — Position, rotation ve scale gibi uzaysal dönüşümlerin birlikte temsildir.

Trigger — Başka collider'ı engellemeden giriş/çıkış olayı üreten bölge için genel terim; Godot'ta Area sık karşılığıdır.

Tutorial — Oyuncuya mekanik/sistem öğreten bölüm veya yöntem.

Tween — Bir değeri başlangıçtan hedefe belirli süre/easing ile geçirmek için kullanılan hafif animasyon yaklaşımıdır.

UI — UX'in görsel/etkileşimsel arayüz katmanı; UX ile aynı şey değildir.

UI / GUI — User Interface, oyuncunun bilgi gördüğü ve etkileştiği arayüzdür: buton, panel, menü, HUD vb. GUI çoğu bağlamda grafik arayüz anlamında kullanılır.

Unique Node — Scene içinde %Name benzeri benzersiz ad erişimi için işaretlenen node.

UX — User Experience: kullanıcının sistemi kullanırken yaşadığı bütün deneyim.

Variable (Değişken) — Programın çalışma sırasında tuttuğu isimlendirilmiş veridir. Değeri değişebilir.

Variant — Godot'un pek çok farklı veri tipini taşıyabilen genel değer türü.

Vector2 / Vector3 — Yön, konum farkı, hız gibi birden fazla sayıyı birlikte temsil eden temel matematik türleridir.

Velocity / Acceleration — Velocity hız ve yönü; acceleration velocity'nin zamanla değişimini ifade eder.

Version Control — Dosya değişikliklerinin geçmişini kaydedip karşılaştırma, geri dönme ve ekipçe çalışma sistemidir. Git en yaygın örneklerden biridir.

Vertical Slice — Oyunun küçük ama hedef kaliteyi temsil eden uçtan uca tamamlanmış parçasıdır.

Viewport — Bir sahnenin/görüntünün render edildiği yüzey/alan. Ana pencere de bir viewport mantığı taşır; ayrıca alt viewport'lar yapılabilir.

Viewport Size — Render alanının genişlik/yüksekliği.

VSync — Frame üretimini ekran yenileme döngüsüyle senkronlamayı amaçlayan seçenek.

Warning — Kod çalışabilir ama şüpheli/istenmeyen durum bildiren uyarı.

Weighted Random — Seçeneklerin eşit değil belirli ağırlıklarla seçildiği rastgelelik yöntemidir.

World Environment — Godot 3D'de environment ayarlarını sahneye uygulayan yapı.

World Space — Global/dünya koordinat uzayı.

Z-index — 2D öğelerin önde/arkada çizim sırasını etkileyen değer.

İlk 50 terim kontrol listesi

☐ Game Engine	☐ Editor
☐ Project	☐ Asset
☐ API	☐ Build / Export
☐ Runtime	☐ Variable
☐ Data Type	☐ Function / Method
☐ Parameter vs Argument	☐ Return Value
☐ Conditional	☐ Loop
☐ Array / Dictionary	☐ Class / Object / Instance
☐ Node	☐ Scene
☐ SceneTree	☐ Parent / Child
☐ PackedScene / instantiate()	☐ _ready()
☐ Signal	☐ Autoload
☐ Game Loop	☐ Frame / FPS
☐ _process(delta)	☐ _physics_process(delta)
☐ delta	☐ Input Action / Input Map
☐ Vector2 / Vector3	☐ Normalize
☐ Local vs Global Coordinates	☐ Transform
☐ Collision	☐ CollisionShape2D / 3D
☐ CharacterBody	☐ Area
☐ Collision Layer / Mask	☐ RayCast / ShapeCast
☐ Sprite / Texture	☐ Camera
☐ AnimationPlayer	☐ Control
☐ Container	☐ Anchor / Offset
☐ Resource	☐ Serialization
☐ State	☐ Finite State Machine (FSM)

Mini kontrol cevapları

1.1 Engine temel çalışma sistemlerini sağlar; editor bu sistemlerle proje oluşturduğun görsel araçtır.

1.2 Bir sistemin koddan kullanılabilen yüzeyi/kuralları: sınıflar, metodlar, propertyler ve çağrı biçimleri.

1.3 Save proje kaynaklarını kaydeder; export oyuncuya dağıtılacak hedef platform çıktısı üretir.

2.1 Parameter fonksiyon tanımındaki isim; argument çağrıdaki gerçek değerdir.

2.2 Array sıralı/indexli koleksiyon; Dictionary key-value eşleşmesidir.

2.3 Hataları daha erken yakalamaya ve editor autocomplete/önerilerini iyileştirmeye yardım eder.

- 3.1 Hayır. Player, UI paneli, mermi, düşman, menü gibi tekrar kullanılabilir node ağaçları da scene olabilir.
- 3.2 PackedScene kaynağından runtime'da yeni bir node ağacı/scene instance oluşturur.
- 3.3 Sahneler arası kalıcı global servisler: GameState, AudioManager, save service gibi.
- 4.1 Son güncelleme adımından bu yana geçen zamanı, genellikle saniye cinsinden.
- 4.2 Kodun fiziksel tuşa değil anlamsal action'a bağlanmasını sağlar; remap ve gamepad desteğini kolaylaştırır.
- 4.3 Görsel/frame işleri _process; fizik/movement hesapları çoğunlukla _physics_process.
- 5.1 Sıfır olmayan vector için 1.
- 5.2 Local parent referansına, global dünya/scene referansına göredir.
- 5.3 Kamera takibi, UI değer geçişi, hareket/rotation smoothing gibi iki değer arasında yumuşak geçiş gereken yerlerde.
- 6.1 Area algılama/trigger için; StaticBody fiziksel sabit engel/çarpışma gövdesi için kullanılır.
- 6.2 Layer nesnenin ait olduğu kategori; mask hangi kategorileri kontrol edeceğini söyler.
- 6.3 Zemin kontrolü, görüş hattı, hitscan silah, duvar algılama vb.
- 7.1 Texture görüntü verisi; Sprite bunu sahnede gösteren nesne/node.
- 7.2 Shader çizim algoritması/mantığı; material bu shader'ı ve parametrelerini belirli yüzeyde kullanan kaynak.
- 7.3 Tek/az sayıda değeri kısa süreli easing ile programatik geçirmek gerektiğinde.
- 8.1 Resolution piksel boyutu; aspect ratio genişlik/yükseklik oranı.
- 8.2 Child UI öğelerinin yerleşim ve boyutunu otomatik yöneterek responsive düzeni kolaylaştırır.
- 8.3 Ortak görsel kuralları merkezileştirir ve tutarlı/değiştirilebilir tasarım sağlar.
- 9.1 Resource paylaşılabılır veri/asset; Scene node ağacı ve davranışsal/uzamsal yapı için uygundur.
- 9.2 Sistemler aşırı bağlıysa birindeki değişiklik diğerlerini kolayca kırar; düşük coupling değişimi/testi kolaylaştırır.
- 9.3 Bir nesnenin farklı durumlarını ve aralarındaki geçiş kurallarını açık biçimde yönetmeye.
- 10.1 Mechanic tekil kural/eylem; core loop oyuncunun tekrar tekrar yaptığı birden çok adımlı ana döngü.
- 10.2 Aynı rastgele diziyi tekrar üreterek test, replay veya deterministik senaryoları kolaylaştırabilir.
- 10.3 Hayır; sadece hedefe rota bulma problemidir. Karar verme ayrı bir AI davranış problemidir.
- 11.1 Remote Procedure Call; ağ üzerinden başka peer'da bir işlemi/fonksiyonu tetikleme mekanizmasıdır.
- 11.2 Hayır. FPS yerel render/güncelleme hızı; latency ağ gecikmesidir.
- 11.3 Bir oyun durumunda hangi tarafın doğru/kontrol sahibi olduğunu belirleyerek tutarlılık ve güvenliği yönetir.
- 12.1 Kod geçerli şekilde çalışır ama tasarlanan sonucun dışında davranır.
- 12.2 Gerçek performans maliyetini ve bottleneck'i ölçmek.
- 12.3 Hata noktasına hangi fonksiyon çağrı zinciriyle geldiğini.
- 13.1 Belirli bir değişiklik grubunun version control geçmişindeki kayıt noktasıdır.
- 13.2 Hayır. Version control değişiklik geçmişi ve işbirliği sağlar; backup veri kaybına karşı ayrı kopyadır.
- 13.3 Debug geliştirme/hata ayıklama odaklı; release dağıtım ve optimize edilmiş çalışma odaklıdır.
- 14.1 Prototype fikri hızlı test eder; vertical slice küçük ama hedef üretim kalitesini uçtan uca temsil etmeye çalışır.
- 14.2 Planlanan iş kapsamının kontrolsüz biçimde büyümesi.
- 14.3 Hayır. UI arayüz; UX kullanıcının ürünle yaşadığı genel deneyimdir.

Kaynak notu

SÜRÜM NOTU Godot'a özgü terimler Godot 4.x yaklaşımıyla yazıldı. Resmî "stable" dokümantasyon başlıkları Ağustos 2026'da terminoloji kontrolü için gözden geçirildi.

- Nodes and Scenes:** https://docs.godotengine.org/en/stable/getting_started/step_by_step/nodes_and_scenes.html
- GDScript reference:** https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/gdscript_basics.html
- Static typing in GDScript:** https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/static_typing.html
- Using signals:** https://docs.godotengine.org/en/stable/getting_started/step_by_step/signals.html
- Idle and Physics Processing:** https://docs.godotengine.org/en/stable/tutorials/scripting/idle_and_physics_processing.html
- Physics introduction:** https://docs.godotengine.org/en/stable/tutorials/physics/physics_introduction.html
- Using InputEvent / Input:** <https://docs.godotengine.org/en/stable/tutorials/inputs/inputevent.html>
- Using Containers:** https://docs.godotengine.org/en/stable/tutorials/ui/gui_containers.html
- Singletons (Autoload):** https://docs.godotengine.org/en/stable/tutorials/scripting/singletons_autoload.html
- The Profiler:** https://docs.godotengine.org/en/stable/tutorials/scripting/debug/the_profiler.html
- Exporting projects:** https://docs.godotengine.org/en/stable/tutorials/export/exporting_projects.html
- Best practices:** https://docs.godotengine.org/en/stable/tutorials/best_practices/index.html

TERİMİ TANI. KENDİ CÜMLENLE AÇIKLA. PROJEDE KULLAN.

Bu rehber bitince ne yapmalısın?

1. KARŞILAŞMA Terimi ilk kez duyduğunda sözlükten bul ve hangi kategoriye ait olduğunu öğren.

2. KARŞILAŞMA Kendi cümlele 1 cümle tanım ve 1 kullanım örneği üret.

3. KARŞILAŞMA Godot'ta küçük bir denemede kullan. O noktadan sonra terim gerçekten senin olur.

Bu PDF bir "bitirilecek kitap" değil, kurs boyunca geri döneceğin masaüstü referansın.